

**INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY
CAMPUS MONTERREY**



**Diseño de sistemas en chip - TE2002B
Grupo: 402**

Etapas #3. Entrega Final

Roberto Raymundo Gómez Vargas - A01285451

Jesus Alberto García Gonzalez - A01369587

Rosendo De Los Rios Moreno-A01198515

Hugo Daniel Castillo Ovando - A00836025

Docentes:

Enrique González Guerrero

Raúl Peña Ortega

I. Introducción.....	4
II. Justificación del problema.....	5
III. Metodología.....	5
IV. Juego.....	7
A. Objetivo.....	7
B. Explicación de Mundos / Escenas.....	7
Figura 1. Mundos / Escenas en Unity.....	7
Tabla 1. Ilustraciones de las escenas y mundos.....	11
C. Personajes.....	12
Tabla 2. Ilustración del personaje.....	12
D. Reglas del Juego.....	12
E. Dinámicas.....	13
Figura 2. Diseño de tarjeta FPGA DE10-Lite.....	13
V. Máquina de Estados (HLMS)S.....	15
Figura 3. Diagrama máquina de estados.....	15
A. Representación de estados para la secuencia del videojuego.....	15
B. Indicación de estado inicial.....	16
C. Transiciones entre estados.....	16
D. Entradas, salidas y variables en formato bit y multi-bit.....	16
E. Uso de expresiones aritméticas para la asignación de variables y salidas.....	17
F. Operaciones de igualdad y relacional para las transiciones entre estados.....	17
VI. Video demostrativo Etapa 1.....	17
VII. Programación Placa.....	18
A. Funcionalidad de Hardware FPGA.....	18
Figura 4. Código implementación de switches.....	19
Figura 5. Código implementación de botones.....	20
Figura 6. Código implementación de display.....	21
Figura 7. Código comunicación con Unity 1/2.....	22
Figura 8. Código comunicación con Unity 2/2.....	23
2. Pruebas de funcionamiento.....	24
Figura 9. Variables para manejar el primer switch.....	24
Figura 10. Condicional para el primer switch.....	25
Figura 11. Se actualizan las variables auxiliares.....	26
Figura 12. Resultado de activar el primer switch.....	27
Figura 13. Variables para manejar el primer botón.....	28
Figura 14. Condicional para el primer botón.....	29
Figura 15. Se actualizan las variables auxiliares.....	30
Figura 16. Se actualizan las variables auxiliares.....	31
Figura 17. Variable de entrada.....	32
Figura 18. Manejo de dato de entrada.....	32
Figura 19. Implementación de “case”.....	33

Figura 20. Visualización del display en tiempo real.....	34
VIII. Conexión Serial.....	35
1. Conexión entre la placa y el juego de Unity.....	35
Figura 21. Función de start con configuración de Serial Port.....	35
Figura 22. Función de Update, con inicialización del contador de Corn.....	36
Figura 23. Función de SendDataToSerialPort con condicionales para envío de datos...	36
Figura 24. Función de ControlVehicleWithSerial.....	37
Figura 25. Función ParseSerialData con las condiciones de los valores de data, para manejo de funciones aplicadas.....	38
Figura 26. Declaración de funciones, como boost, break, turnleft y turnright, y su respectiva desactivación.....	39
2. Pruebas de funcionamiento.....	40
Figura 27. Implementación de envío de recepción de datos a través de la consola de Unity.....	40
Figura 28. Tercer switch activado.....	41
Figura 29. Implementación de los botones y su respectivo valor serial visto desde la consola.....	43
Figura 30. Funcionamiento de los botones, y su respectivo valor serial.....	44
Figura 31. Recepción de datos dentro de la consola.....	45
Figura 32. Despliegue del Corn Counter en el FPGA.....	46
IX. Video Demostrativo Etapa 2.....	48
1. Funcionamiento.....	49
Figura 33. Funcionamiento de la FPGA.....	49
2. Acelerómetro y Procesador.....	50
Figura 34. Proceso TX_Start y Botón 1.....	50
Figura 35. Proceso botón 2 y swiches.....	51
Figura 36. Proceso acelerómetro, TX_Data y Select.....	52
Figura 37. Etiquetas principales.....	53
Figura 38. Proceso para checar por el botón 1.....	54
Figura 39. Proceso para checar por el switch 1.....	55
Figura 40. Proceso para checar por el switch 2.....	56
Figura 41. Proceso para checar por el botón 2.....	57
Figura 42. Proceso para checar por el acelerómetro 1/2.....	58
Figura 43. Proceso para checar por el acelerómetro 2/2.....	59
Figura 44. Proceso del delay.....	60
3. Diseño RTL.....	61
Figura 45. Diseño RTL completo.....	61
Figura 46. Sección más importante del diseño RTL.....	62
4. Video demostrativo prototipo final.....	62
5. Conclusiones.....	63
XI. Bibliografía.....	64
XII. Anexos.....	65

I. Introducción

En este proyecto, nuestro equipo empleará un PLD (Dispositivo Lógico Programable) dentro de una FPGA (Field Programmable Gate Array), justificado por su versatilidad, capacidad de integración, eficiencia y facilidad de mantenimiento. Esta elección nos permitirá desarrollar la lógica de control del tractor, interpretar señales y gestionar salidas de manera adaptable a diversas situaciones. La capacidad de reprogramación de la FPGA facilitará ajustes y correcciones, asegurando la confiabilidad a largo plazo del sistema. Además, se implementará el desarrollo de una simulación en Unity. Esta simulación permitirá simular el funcionamiento del tractor en un entorno virtual, con el fin de probar y validar el sistema, esto aumentará la eficiencia y reducirá los riesgos durante el proceso de desarrollo.

Es crucial considerar el contexto más amplio de la agricultura moderna, donde iniciativas como la agricultura de precisión y la implementación de tecnologías como el Internet de las Cosas (IoT) están transformando el sector agrícola. La agricultura de precisión emplea tecnologías avanzadas como sensores, GPS y análisis de datos para optimizar el uso de recursos vitales como el agua, los fertilizantes y los pesticidas. Esta estrategia no solo reduce el desperdicio de recursos, sino que también impulsa la productividad agrícola. Desde una perspectiva ética, promueve una gestión más responsable de los recursos naturales al minimizar el uso excesivo de productos químicos, contribuyendo así a la reducción del impacto ambiental. Sin embargo, la recopilación masiva de datos agrícolas plantea desafíos en términos de privacidad y seguridad de la información, y el acceso equitativo a estas tecnologías puede ser un obstáculo para los agricultores más pequeños o con menos recursos.

II. Justificación del problema

Nuestro enfoque en el desarrollo del juego en Unity se basa en utilizar una tecnología llamada PLD dentro de una FPGA para controlar los movimientos del tractor y mostrar información importante en la pantalla. Esta elección es muy importante porque nos permite crear una experiencia de juego realista.

Ya que si se llega a manipular el juego, se tendría el control total del tractor. Gracias a la PLD y la FPGA, se puede simular con precisión cómo se mueve y responde a las acciones y verlo reflejado en el Unity. Esto hace que el juego sea más realista y mejora la experiencia. Además, en la pantalla, se podrá desplegar información útil como contadores y datos relevantes. Lo que dará una visión más clara de lo que está sucediendo en el juego. Y de esta manera, se podrá tomar decisiones informadas y entender cómo las acciones afectan el rendimiento del tractor y los resultados del juego.

En el contexto de la agricultura moderna y la simulación de sistemas en entornos virtuales, la utilización de un PLD dentro de una FPGA para controlar los movimientos del tractor y mostrar información mediante Unity es crucial. Esta elección se fundamenta

en la necesidad de optimizar la gestión de recursos agrícolas y reducir costos, así como en la capacidad de replicar con precisión situaciones del mundo real para evaluar el desempeño del tractor. La integración de la funcionalidad de la FPGA en la simulación en Unity no solo mejora la precisión de los resultados, sino que también reduce los riesgos asociados con el desarrollo de hardware físico, mientras que la visualización de datos en pantalla proporciona una herramienta valiosa para la toma de decisiones informadas y el entrenamiento de operadores agrícolas. Todo esto con el objetivo de buscar mejorar los tractores de John Deere donde sus clientes sean beneficiados con un mejor producto y que la empresa obtenga mejores márgenes de ganancia.

III. Metodología

La metodología que proponemos para este proyecto se basa en una serie de pasos que nos permitirán implementar un sistema de control para tractores agrícolas. En primer lugar, se utiliza una FPGA con un PLD. El cual nos permitirá adaptar la lógica de control a diferentes situaciones, de manera eficiente y fácil de mantener. Además, vamos a utilizar Unity, una plataforma de simulación, para recrear el tractor y su entorno de trabajo en un entorno virtual. Esto nos permitirá tener una representación realista del tractor y también aprovechar las capacidades avanzadas de Unity en gráficos y física.

Una vez que tengamos las herramientas, vamos a integrar los controles del juego con la placa FPGA DE10-Lite. Por ejemplo, vamos a utilizar el acelerómetro de la placa para simular la dirección del tractor, botones para el acelerador y el freno, y displays para mostrar la cantidad de cosecha recolectada. Después de la integración de los controles del juego con la placa, vamos a desarrollar la lógica de control del tractor. Esto implica diseñar y programar el PLD dentro de la FPGA para interpretar las señales de entrada, procesar los datos y controlar los movimientos del tractor.

Durante la etapa de integración con Unity, dividimos el proceso en diferentes fases principales. En la primera, nos centramos en diseñar entornos y establecer las físicas adecuadas. Comenzamos adaptando un objeto tractor importado desde Unity para que reflejara fielmente el espacio físico del vehículo. Desactivamos su representación visual y asignamos propiedades físicas como “wheelCollider” para las llantas. Además, etiquetamos las plantaciones de maíz para interactuar con los scripts correspondientes, como “CornCounter.cs” y “PrometousCarControlleer.cs”.

En cuanto a la configuración de la Placa FPGA, utilizamos Quartus Prime Lite Edition para desarrollar el código VHDL necesario. Esto permitió establecer la comunicación serial entre la FPGA y el simulador en Unity. Además, facilitó la interacción con el display de 7 segmentos, 4 switches y 2 botones. Estas funcionalidades posibilitaron el control del tractor y la visualización en tiempo real de la cantidad de maíz recolectado, todo sincronizado con la simulación en Unity.

IV. Juego

A. Objetivo

El objetivo de este proyecto es utilizar la tarjeta de prototipo DE10-Lite para poder implementar los movimientos del tractor en la tarjeta programable FPGA. Con esto se debe de lograr una simulación de la operación del movimiento del tractor y visualizarlo mediante un programa en Unity que sirva para ver el desarrollo del ambiente del tractor. El programa en Unity funciona solo como display y no debe de procesar los datos.

B. Explicación de Mundos / Escenas

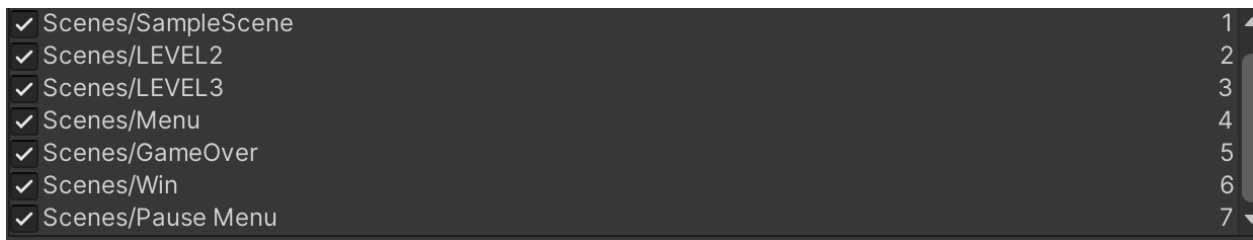


Figura 1. Mundos / Escenas en Unity

- Escenas

- Menú Principal

El Menú Principal es la primera pantalla del juego donde el jugador puede seleccionar su personaje y comenzar el juego.

- Menú de pausa

El Menú de Pausa puede ser accedido en cualquiera de los tres niveles. Aquí, el jugador tiene la opción de regresar al nivel en el que estaba jugando o volver al Menú Principal y dejar de jugar.

- Pantalla de victoria

Si el jugador logra completar el juego exitosamente, se presenta la escena de Pantalla de Victoria, indicando que el jugador ha ganado.

- Pantalla de derrota

Por otro lado, si el jugador pierde el juego, se presenta la escena de Pantalla de Derrota, indicando que el jugador ha perdido.

- Sample Scene/ Nivel 1

Primer nivel de el juego donde el tractor debe de recolectar máximo 50 plantas de maíz y evitar rocas las cuales dañan su funcionamiento, el tractor tiene una vida de 100% y al momento de que 3 rocas lo golpeen, se llegará al 0%, esto te mandaria a la escena de derrota.

→ Escena/ Nivel 2

Primer nivel de el juego donde el tractor debe de recolectar máximo 100 plantas de maíz y evitar rocas las cuales dañan su funcionamiento, el tractor tiene una vida de 100% y al momento de que 3 rocas lo golpeen, se llegará al 0%, esto te mandaria a la escena de derrota.

→ Escena/ Nivel 3

Primer nivel de el juego donde el tractor debe de recolectar máximo 150 plantas de maíz y evitar rocas las cuales dañan su funcionamiento, el tractor tiene una vida de 100% y al momento de que 3 rocas lo golpeen, se llegará al 0%, esto te mandaria a la escena de derrota.

- Mundos

La simulación se divide en tres niveles, que el jugador deberá completar para ganar el juego:

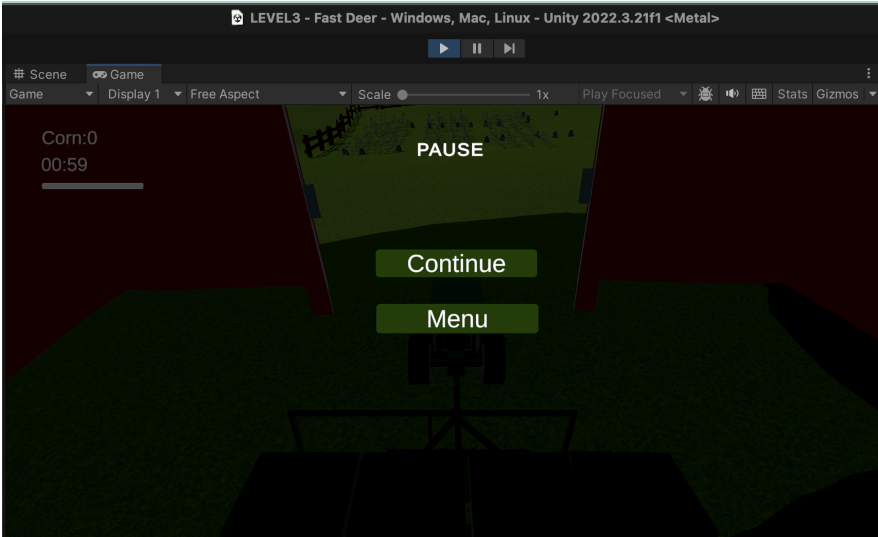
→ Nivel 1

→ Nivel 2

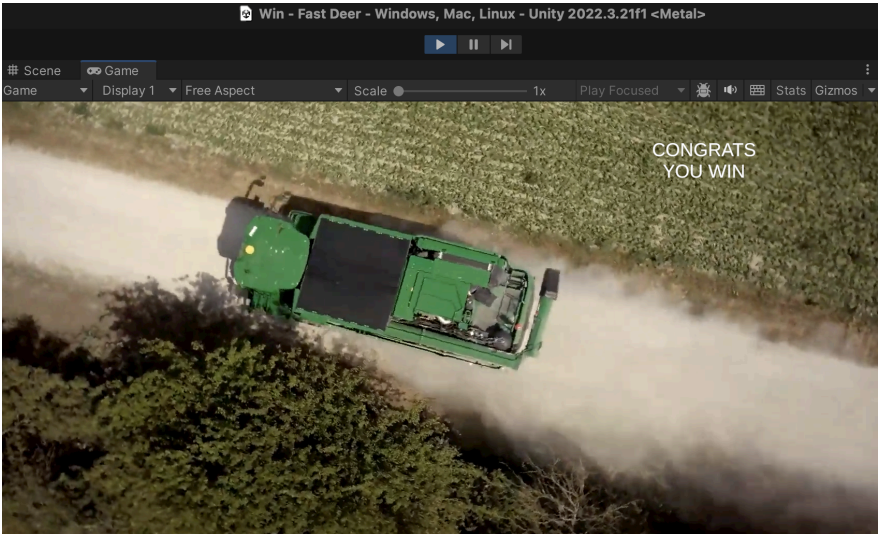
→ Nivel 3



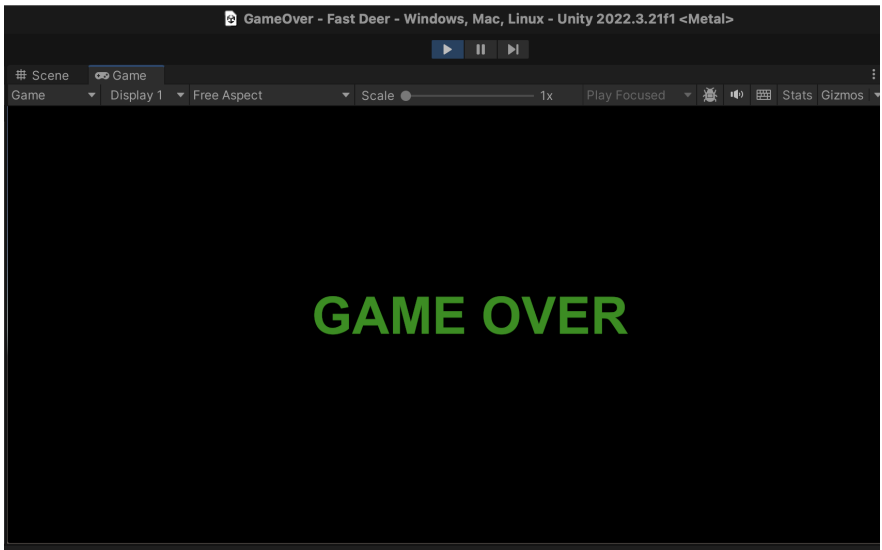
Menú de pausa



Pantalla de victoria

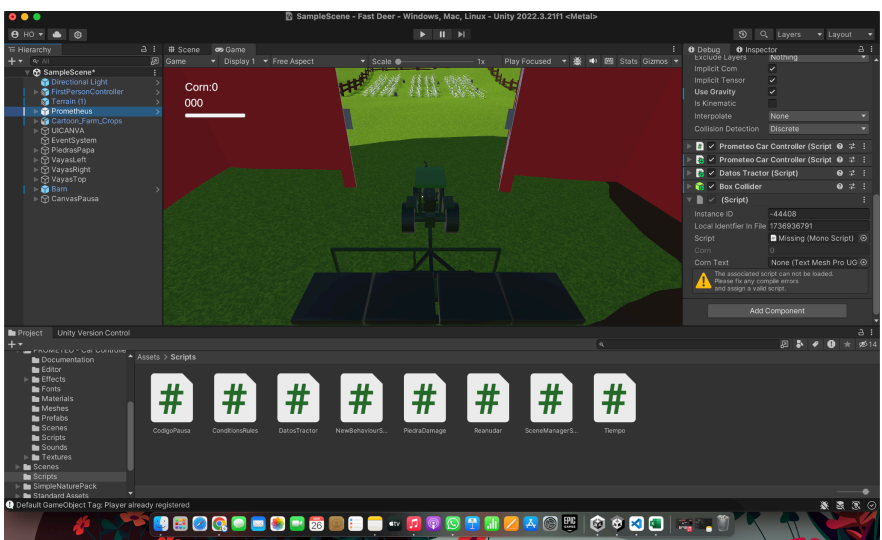


Pantalla de derrota



Mundos

Nivel 1



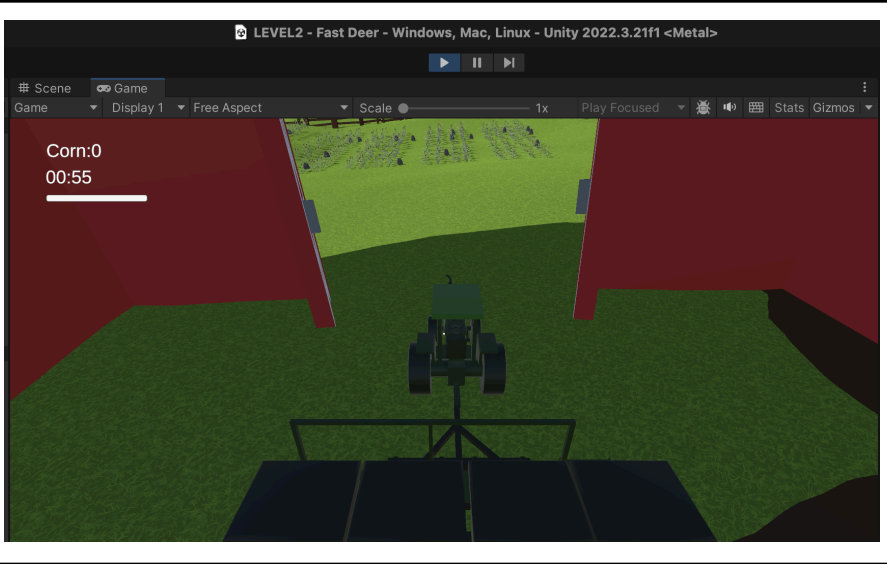
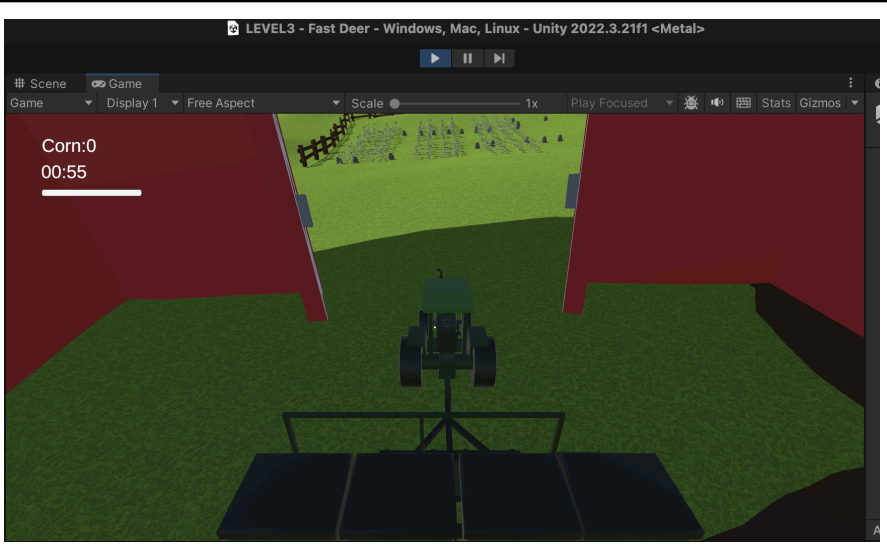
Nivel 2	
Nivel 3	

Tabla 1. Ilustraciones de las escenas y mundos

C. Personajes

En nuestro juego, contamos con el personaje del tractor, el cual está representado en un color verde, emulando la estética característica de un tractor John Deere. Asimismo, el tractor está equipado con los dispositivos adicionales que fueron discutidos durante la reunión con el socio formador, los cuales se encuentran ubicados en la parte inferior del tractor. Estos dispositivos están diseñados para replicar las funcionalidades que se añaden a un tractor John Deere para mejorar y ampliar sus capacidades.

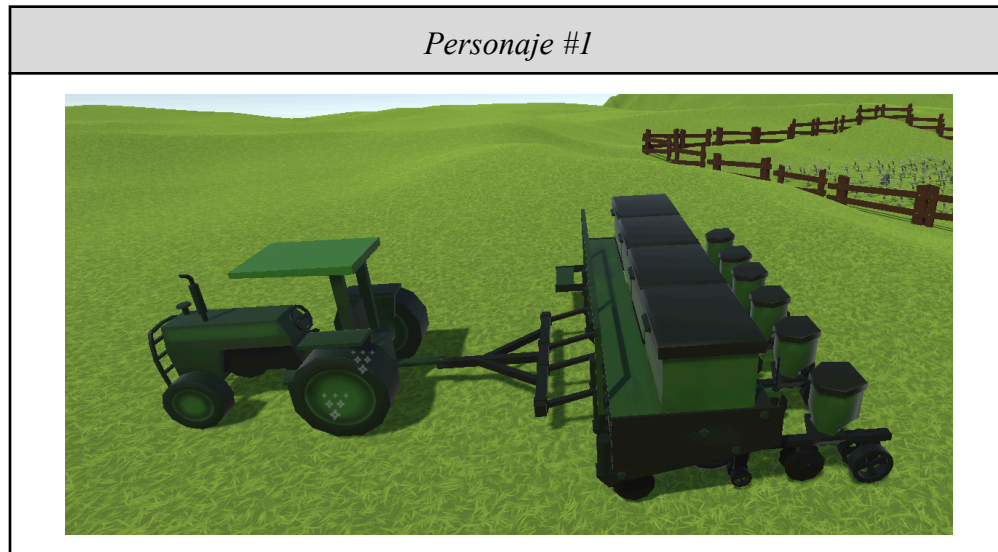


Tabla 2. Ilustración del personaje

D. Reglas del Juego

Nuestro juego consta de tres desafiantes niveles. El objetivo es completar el nivel 3 antes de que el tiempo se agote y sin perder el porcentaje de vida. El jugador se encuentra en un entorno de campo donde debe cosechar cultivos. El límite de tiempo es de 60 segundos.

El primer nivel permite cosechar entre 0 y 50 elementos. Al superar los 50 elementos, se avanza al segundo nivel. En el segundo nivel al llegar a los 100 elementos marca el comienzo del tercer nivel, y en el tercer nivel al momento de llegar a los 150 elementos, el juego concluye con una pantalla de victoria.

Cada nivel presenta obstáculos en forma de rocas; cuanto más avanzado sea el nivel, más obstáculos habrá. Chocar contra un obstáculo resulta en la pérdida de un porcentaje de vida, la vida se puede ver marcada por una Barra que va de 0 a 100 de vida, al momento de chocar va bajando la barra y esta se acaba al momento de chocar con tres rocas. Al momento de que la barra se acabe o agotar los 60 segundos lleva al final del juego, con la consiguiente se desplegará la derrota del jugador.

E. Dinámicas

Relación del juego con la placa

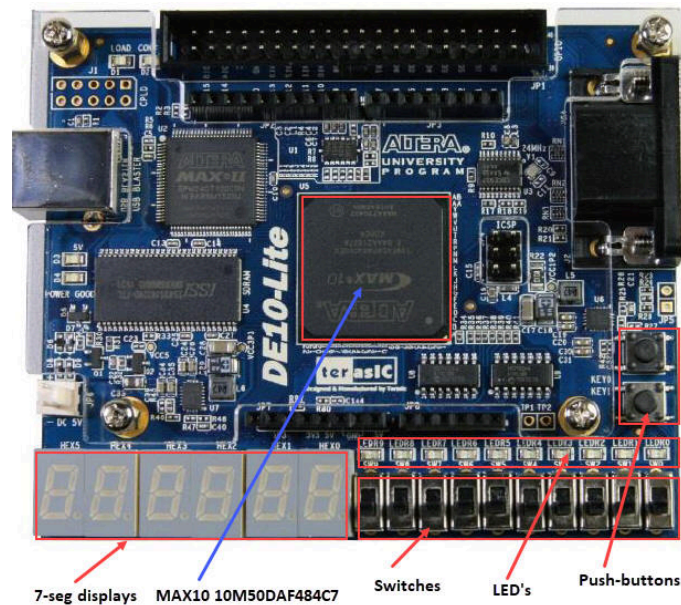


Figura 2. Diseño de tarjeta FPGA DE10-Lite

Para desarrollar la simulación del tractor se utilizaron los elementos ya disponibles dentro de la tarjeta DE10-Lite.

1. Manejo → Acelerómetro

“Steering”

Para poder conducir el tractor, y poder direccionar a la derecha o a la izquierda, se utilizará el acelerómetro ya integrado dentro de la tarjeta para poder simular el uso de un volante.

2. Selección de marcha → Switches

“Gear selection”

Para poder seleccionar la marcha del tractor, en marcha o en reversa, se utilizarán dos switches los cuales nos permitirá tener 4 opciones. Con la combinación de los switches se seleccionará en marcha, en reversa o en neutro.

3. Acelerador → Botón

“Throttle”

Para poder simular el acelerar el tractor se usará un botón, el cual mientras sea presionado el tractor aumenta de velocidad y al dejar de ser presionado dejará de aumentar de velocidad.

4. Freno → Botón

“Brake”

Del mismo modo que acelerar, se utilizara un botón para simular el frenado del tractor. Mientras el botón sea presionado, el tractor disminuirá la velocidad, y al dejar de ser presionado el tractor dejará de frenar.

5. Contador de objetos recolectados → (Display de 7 segmentos)

Nuestro juego tiene como propósito que el usuario recolecta 150 elementos de cosecha, utilizando el display de 7 segmentos desplegamos la cantidad de cosecha que el jugador tenga recolectada, así que cada vez que recolectó nueva cosecha se aumentará el contador en el display.

V. Máquina de Estados (HLMS)S

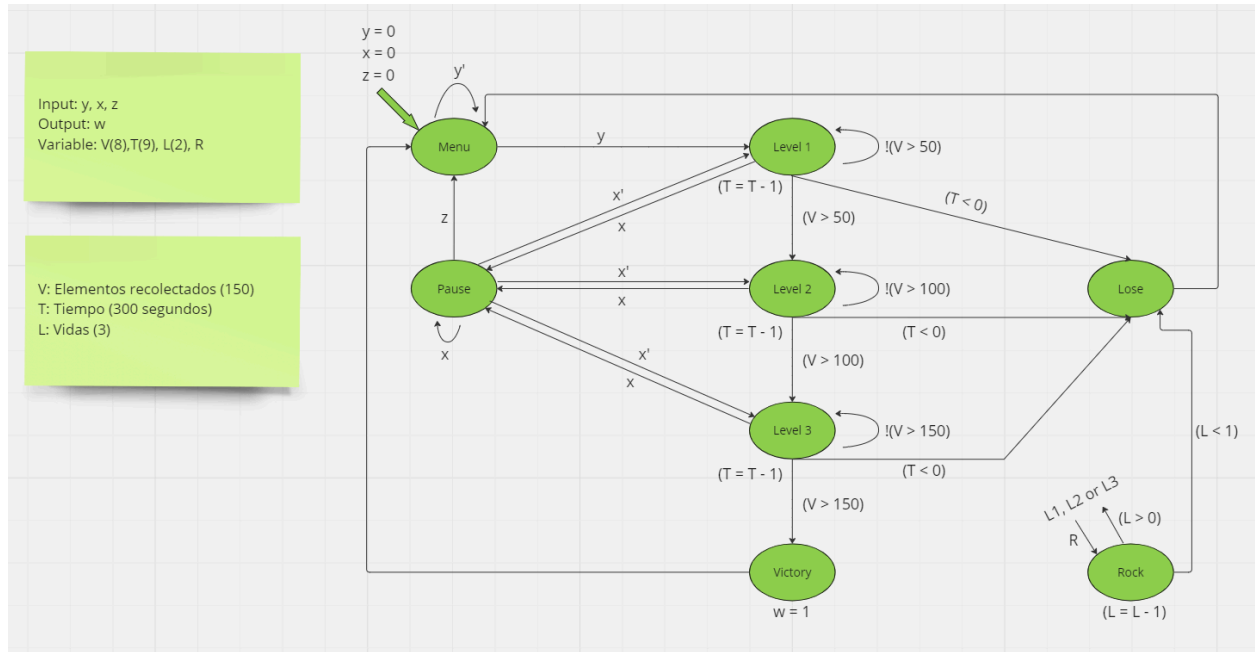


Figura 3. Diagrama máquina de estados

A. Representación de estados para la secuencia del videojuego

Nuestra simulación del tractor comprende ocho estados distintos. Los tres principales son los estados correspondientes a los diferentes niveles, con un estado designado para cada nivel del uno al tres. Además de estos, contamos con estados de menú: el menú principal, que actúa como el estado inicial del juego, y el estado de pausa, al que se puede acceder desde cualquier nivel en el que el jugador esté activo.

Posteriormente, encontramos los estados que determinan la finalización del juego: el estado de pantalla de victoria y el estado de pantalla de derrota. Ambos estados redirigen automáticamente al menú principal después de un tiempo predefinido.

Finalmente, está el estado de acción, conocido como el estado "Rock". Este estado gestiona la pérdida de una vida del jugador en caso de impacto con una roca. Si se pierde el porcentaje de vida, este estado lleva a la pantalla de derrota; de lo contrario, regresa al estado del nivel con la vida disponible.

B. Indicación de estado inicial

El estado inicial del juego es el "Menú", donde se encuentra el menú principal del juego y donde el jugador puede seleccionar su personaje y comenzar a jugar. En este estado, las variables 'x', 'y' y 'z' se inicializan en 0. Cuando el jugador selecciona la opción de jugar dentro de este estado principal, será dirigido al primer nivel del juego, donde podrá comenzar a cosechar el terreno.

C. Transiciones entre estados

Las transiciones entre los estados están determinadas por las variables 'x', 'y' y 'z'. La variable 'y' controla la transición entre el menú principal y el nivel uno: mientras 'y' esté desactivada, el jugador permanecerá en el menú principal; cuando se active 'y', el jugador será enviado al estado del nivel 1.

Las transiciones entre los tres niveles dependen de la variable 'V', que aumenta a medida que el jugador recolecta elementos de cosecha, lo que indica el progreso de nivel. En el nivel uno, 'V' va de 0 a 50; en el nivel dos, de 50 a 100; y en el nivel tres, de 100 a 150. Una vez que 'V' supera 150, se produce la transición al estado de pantalla de victoria.

El estado de 'Roca' se activa mediante la variable 'R'. Cuando 'R' se activa, el jugador pasa al estado de "Roca" temporalmente y luego regresa al nivel correspondiente, o bien, si no quedan vidas, al estado de derrota. Si la variable 'T' alcanza 0, el jugador será enviado al estado de derrota.

Finalmente, si se activa la variable 'x' en cualquier nivel, se enviará al jugador al estado de "pausa". Al desactivar 'x', el jugador volverá al nivel correspondiente; sin embargo, si se activa la variable 'z' mientras está en el estado de pausa, se lo enviará de regreso al menú principal.

D. Entradas, salidas y variables en formato bit y multi-bit

→ Entradas (*bit*): y, x, z

→ Salidas (*bit*): w

→ Variables (*bit*): R

→ Variables (*multi-bit*): V, T, L

- V = 150 (8 bits) ; Elementos de cosecha recolectados
- T = 300 (9 bits) ; Tiempo
- L = 3 (2 bits) ; Vidas

E. Uso de expresiones aritméticas para la asignación de variables y salidas

En la máquina de estados presente en nuestro juego, se requiere el uso de expresiones aritméticas para la asignación de variables. En cada ciclo del reloj, en los estados del nivel 1 al nivel 3, se utiliza la expresión aritmética " $T = T - 1$ " para decrementar el tiempo disponible. Del mismo modo, cada vez que se recolecta un elemento de cosecha, se emplea la expresión aritmética " $V = V + 1$ " para aumentar el contador correspondiente al progreso del nivel. Asimismo, cuando una roca es impactada, se utiliza la expresión " $L = L - 1$ " para restar una vida del jugador.

F. Operaciones de igualdad y relacional para las transiciones entre estados

En la máquina de estados presente en nuestro juego, se requiere el uso de operaciones de igualdad y relaciones para las transiciones entre estados. Inicialmente, se emplean los operadores ' $>$ ' y ' $<$ ' en combinación con la variable ' V ' para determinar cuándo se debe avanzar de un estado de nivel a otro. Se utiliza " $V > \dots$ " para indicar cuándo es momento de transicionar al siguiente nivel, y se usa " $!(V > \dots)$ " para determinar si aún se debe permanecer en el mismo estado.

De igual modo, se utiliza la operación relacional con el tiempo para determinar cuándo el juego ha terminado y el jugador ha perdido. Esto se logra mediante la expresión " $T < 0$ ", que se encuentra presente en los tres niveles. Una vez que esta condición se cumple, el juego finaliza y el jugador pierde.

Finalmente, en el estado de "Rock", se utilizan las operaciones " $L > 0$ " y " $L < 1$ ". La segunda lleva al jugador a la pantalla de derrota, mientras que la primera lo devuelve al nivel correspondiente.

VI. Video demostrativo Etapa 1

<https://youtu.be/aSYh-YAftXQ?si=55NUkW3I-6bF4HaM>

VII. Programación Placa

A. Funcionalidad de Hardware FPGA

En nuestra implementación de la simulación, mediante la tarjeta FPGA DE10-Lite, se utilizó los elementos ya disponibles dentro de la tarjeta para controlar el tractor y visualizar los datos clave. Nuestro proyecto emplea el uso de: cuatro switches, dos botones, y el display de siete segmentos.

1. Función

a. Switches

Mediante los primeros 4 switches se logró ejecutar ocho comandos, los cuatro switches nos permiten mover el tractor. Cada switch cumple con un movimiento, donde al activar el switch se envía la instrucción de ejecutar la acción y al desactivar el switch se envía otra instrucción para detener la ejecución de esa acción.

El primer switch al ser activado mueve el tractor hacia adelante y al ser desactivado detiene esa acción. Esto se logra enviando un número clave que se recibe y se utiliza en el código en Unity donde cada número tiene una acción. Al activar el primer switch se envía el número '1' a Unity y al desactivarlo se envía el número '2'.

El segundo switch al ser activado mueve el tractor hacia atrás y al ser desactivado detiene esa acción. Al activar el segundo switch se envía el número '3' a Unity y al desactivarlo se envía el número '4'.

El tercer switch al ser activado mueve el tractor hacia la derecha y al ser desactivado detiene esa acción. Al activar el tercer switch se envía el número '5' a Unity y al desactivarlo se envía el número '6'.

El cuarto switch al ser activado mueve el tractor hacia la izquierda y al ser desactivado detiene esa acción. Al activar el cuarto switch se envía el número '7' a Unity y al desactivarlo se envía el número '8'.

```
-- Avansar
-- SW = 1
ELSIF(SW = "0000000001" and sw1_past = "0000000000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000001";
-- SW = 0
ELSIF(SW = "0000000000" and sw1_stop_past = "0000000001") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000010";

-- Atras
-- SW = 2
ELSIF(SW = "0000000010" and sw2_past = "0000000000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000011";
-- SW = 0
ELSIF(SW = "0000000000" and sw2_stop_past = "0000000010") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000100";

-- Derecha
-- SW = 3
ELSIF(SW = "0000000100" and sw3_past = "0000000000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000101";
-- SW = 0
ELSIF(SW = "0000000000" and sw3_stop_past = "0000000100") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000110";

-- Izquierda
-- SW = 4
ELSIF(SW = "0000001000" and sw4_past = "0000000000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000111";
-- SW = 0
ELSIF(SW = "0000000000" and sw4_stop_past = "0000001000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001000";
```

Figura 4. Código implementación de switches.

Como se puede ver en la figura anterior, cada switch requiere de utilizar dos “ELSIF” esto logra enviar una instrucción tanto cuando se enciende el switch como cuando se apaga. En general, cada condicional funciona de la siguiente manera. Cuando se activa el switch, se pregunta cuál de los switches fue activado y se pregunta si el estado anterior de todos los switches era apagado, si se cumple con esto entonces se envía el número clave al programa en Unity. Por otro lado, las condicionales de los switches apagados, preguntan el estado de los switches y el estado anterior, si el estado de los switches están apagados y el estado anterior se encuentra el switch proveniente prendido, entonces se envía el número clave al programa en Unity.

b. Botones

La FPGA tiene dos botones disponibles para usar. En nuestro proyecto el primer botón funciona como un acelerador, el cual aumenta exponencialmente la velocidad del tractor. Al ser oprimido, el botón envía el número clave '9' a Unity y al dejar de ser oprimido envía el número "10". El segundo botón se utiliza como un freno de mano, lo cual ocasiona que el tractor frene fuertemente, el botón al ser oprimido envía el número clave "11" y al dejar de ser oprimido envía el número "12" a Unity.

```
-- Boton 1
IF key1_db = '1' and key1_db_past = '0' THEN
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001001";

-- Boton 1
ELSIF key1_db = '0' and key1_db_past = '1' THEN
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001010";

-- Boton 2
ELSIF key2_db = '1' and key2_db_past = '0' THEN
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001011";

-- Boton 2
ELSIF key2_db = '0' and key2_db_past = '1' THEN
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001100";
```

Figura 5. Código implementación de botones

De la misma manera en la que se implementó el comportamiento de los switches los dos botones requieren de dos condicionales cada uno, donde la primera condicional evalúa si el botón fue oprimido y la segunda condicional evalúa si el botón ya no está oprimido. La lógica funciona de esta manera: Se pregunta por el estado del botón si el estado pasado es contrario al estado actual, se envía el número clave al programa en Unity.

c. Display de siete segmentos

En el display se utilizan los primeros 3 segmentos para poder desplegar el contador de elementos recolectados por el tractor. Ya que nuestro juego tiene un número máximo de 150 elementos a cosechar, solo se requiere de utilizar tres segmentos del display. El display se actualiza en tiempo real para que el jugador pueda ver cuantos elementos ha cosechado. Esto se logra al poder recibir un dato desde Unity hasta el código de VHDL en Quartus Prime al pasar por el FTDI y la FPGA. Unity envía el valor en un byte el cual se recibe en el código de VHDL, este dato luego se guarda en tres variables, una variable para cada dígito del dato completo. Una variable guarda la centena del dato, otra variable guarda la decena y otra variable guarda el dígito. Al tener estas tres variables se pueden usar para asignarle a cada una uno de los tres displays. Como resultado tenemos el valor del contador en Unity desplegado en la tarjeta de FPGA.

```
-- Guardar la variable recibida a binario
int_value := to_integer(unsigned(rx_data_de10(7 downto 0)));

-- Extrae centenas, decenas y un dígito.
hundreds := int_value / 100; -- Hundreds place
tens      := (int_value / 10) mod 10; -- Tens place
single    := int_value mod 10; -- Single place

-- Asignar valores de visualización basados en dígitos

-- Primer segmento
case single is
when 0 => Disp_1 <= "1000000"; -- 0
when 1 => Disp_1 <= "1111001"; -- 1
when 2 => Disp_1 <= "0100100"; -- 2
when 3 => Disp_1 <= "0110000"; -- 3
when 4 => Disp_1 <= "0011001"; -- 4
when 5 => Disp_1 <= "0010010"; -- 5
when 6 => Disp_1 <= "0000010"; -- 6
when 7 => Disp_1 <= "1111000"; -- 7
when 8 => Disp_1 <= "0000000"; -- 8
when 9 => Disp_1 <= "0010000"; -- 9
when others => Disp_1 <= "1000000"; -- 0 by default
end case;

-- Segundo segmento
case tens is
when 0 => Disp_2 <= "1000000"; -- Blank leading zero
when 1 => Disp_2 <= "1111001"; -- 1
when 2 => Disp_2 <= "0100100"; -- 2
when 3 => Disp_2 <= "0110000"; -- 3
when 4 => Disp_2 <= "0011001"; -- 4
when 5 => Disp_2 <= "0010010"; -- 5
when 6 => Disp_2 <= "0000010"; -- 6
when 7 => Disp_2 <= "1111000"; -- 7
when 8 => Disp_2 <= "0000000"; -- 8
when 9 => Disp_2 <= "0010000"; -- 9
when others => Disp_2 <= "1000000"; -- 0 by default
end case;

-- Tercer segmento
case hundreds is
when 0 => Disp_3 <= "1000000"; -- Blank leading zero
when 1 => Disp_3 <= "1111001"; -- 1
when 2 => Disp_3 <= "0100100"; -- 2
when 3 => Disp_3 <= "0110000"; -- 3
when 4 => Disp_3 <= "0011001"; -- 4
when 5 => Disp_3 <= "0010010"; -- 5
when 6 => Disp_3 <= "0000010"; -- 6
when 7 => Disp_3 <= "1111000"; -- 7
when 8 => Disp_3 <= "0000000"; -- 8
when 9 => Disp_3 <= "0010000"; -- 9
when others => Disp_3 <= "1000000"; -- 0 by default
end case;
```

Figura 6. Código implementación de display

Como se puede observar en la figura anterior, la variable que se recibe desde el programa de Unity es la variable “rx_data_de10” esta variable se convierte en decimal y sus tres dígitos son guardados en las variables de “hundreds”, “tens” y “single”. Estas tres variables son asignadas a un segmento del display cada una, donde utilizando un case se despliega el número en el display. De esta manera podemos ver el valor del contador de Unity en nuestra tarjeta FPGA

Código completo en el apartado de anexo

```
void ParseSerialData(byte data)
{
    Debug.Log("Dato recibido desde el puerto serial: " + data);

    if (data == 0x01) // Switch 0 arriba
    {
        switch0Activated = true;
        if (!switch1Activated)
        {
            GoForward();
        }
    }
    else if (data == 0x02) // Switch 0 abajo
    {
        switch0Activated = false;
        throttleAxis = 0;
        ApplyMotorTorque(throttleAxis);
    }
    else if (data == 0x03) // Switch 1 arriba
    {
        switch1Activated = true;
        if (!switch0Activated)
        {
            GoReverse();
        }
    }
    else if (data == 0x04) // Switch 1 abajo
    {
        switch1Activated = false;
        throttleAxis = 0;
        ApplyMotorTorque(throttleAxis);
    }
    else if (data == 0x05) // Switch 2 arriba
    {
        switch2Activated = true;
        TurnLeft();
    }
    else if (data == 0x06) // Switch 2 abajo
    {
        switch2Activated = false;
        steeringAxis = 0;
        ApplySteering();
    }
    else if (data == 0x07) // Switch 3 arriba
    {
        switch3Activated = true;
        TurnRight();
    }
}
```

Figura 7. Código comunicación con Unity 1/2.

```

else if (data == 0x08) // Switch 3 abajo
{
    switch3Activated = false;
    steeringAxis = 0;
    ApplySteering();
}

// Esto es lo nuevo
else if (data == 0x09) // boton 1
{
    button1Activated = true;
    boost();
}
else if (data == 0x0A)
{
    button1Deactivated = false;
    DeactivateBoost();
}

else if (data == 0x0B) // boton 2
{
    button2Activated = true;
    breaktractor();
}
else if (data == 0x0C)
{
    button2Deactivate = false;
    DeactivateBreakTractor();
}
}

```

Figura 8. Código comunicación con Unity 2/2

Como se puede observar en el conjunto de imágenes, el programa de Unity está configurado para recibir un número clave mediante la conexión serial, “data”. Este número es procesado por una secuencia de condicionales, donde cada número activa una cierta acción. El código recibe un byte de información y se compara con sus correspondientes en hexadecimal. Empezando con los switches, el número impar activa la acción y el número par desactiva esa acción. Por ejemplo; el número ‘1’ activa la acción de mover hacia adelante y el número ‘2’ desactiva esa acción. De esta manera se configura la implementación de los cuatro swicthes y los dos botones.

2. Pruebas de funcionamiento

a) Prueba 1.

- Acción: Movimiento a la derecha (Switch 1).
- Descripción:
 1. Paso 1: Físicamente se mueve el switch #0 de la tarjeta hacia arriba.
 2. Paso 2: Se utilizarán las variables declaradas previo al comienzo la arquitectura.

```
-- Señales para componente UART
SIGNAL tx_ena_de10:      std_logic := '1';
SIGNAL tx_data_de10:     std_logic_vector(7 DOWNTO 0);
SIGNAL rx_busy_de10:     std_logic;
SIGNAL rx_error_de10:    std_logic;
SIGNAL rx_data_de10:     std_logic_vector(7 DOWNTO 0); -- Contador de corns
SIGNAL tx_busy_de10:     std_logic;

-- Señales para mandar los datos de la placa
-- Boton 1
SIGNAL key1_db:          std_logic;
SIGNAL key1_db_past:     std_logic := '0';

-- Boton 2
SIGNAL key2_db:          std_logic;
SIGNAL key2_db_past:     std_logic := '0';

-- Sw1 Avansar
SIGNAL sw1_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw1_stop_past:    std_logic_vector(9 DOWNTO 0);

-- Sw2 Atras
SIGNAL sw2_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw2_stop_past:    std_logic_vector(9 DOWNTO 0);

-- Sw3 Derecha
SIGNAL sw3_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw3_stop_past:    std_logic_vector(9 DOWNTO 0);

-- Sw4 Izquierda
SIGNAL sw4_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw4_stop_past:    std_logic_vector(9 DOWNTO 0);
```

Figura 9. Variables para manejar el primer switch

3. Paso 3: Proceso para transmitir datos

--Proceso para transmitir datos según las entradas de la interfaz.

```
PROCESS( CLOCK_50 )
BEGIN
    IF rising_edge( CLOCK_50 ) THEN

        -- Boton 1
        IF key1_db = '1' and key1_db_past = '0' THEN
            tx_ena_de10 <= '0';
            tx_data_de10 <= "00001001";

        -- Boton 1
        ELSIF key1_db = '0' and key1_db_past = '1' THEN
            tx_ena_de10 <= '0';
            tx_data_de10 <= "00001010";

        -- Boton 2
        ELSIF key2_db = '1' and key2_db_past = '0' THEN
            tx_ena_de10 <= '0';
            tx_data_de10 <= "00001011";

        -- Boton 2
        ELSIF key2_db = '0' and key2_db_past = '1' THEN
            tx_ena_de10 <= '0';
            tx_data_de10 <= "00001100";

        -- Avansar
        -- SW = 1
        ELSIF(Sw = "0000000001" and sw1_past = "0000000000") then
            tx_ena_de10 <= '0';
            tx_data_de10 <= "00000001";

        -- SW = 0
        ELSIF(Sw = "0000000000" and sw1_stop_past = "0000000001") then
            tx_ena_de10 <= '0';
            tx_data_de10 <= "00000010";

        -- Atras
        -- SW = 2
        ELSIF(Sw = "0000000010" and sw2_past = "0000000000") then
            tx_ena_de10 <= '0';
            tx_data_de10 <= "00000011";

        -- SW = 0
        ELSIF(Sw = "0000000000" and sw2_stop_past = "0000000010") then
            tx_ena_de10 <= '0';
            tx_data_de10 <= "00000100";
```

Figura 10. Condicional para el primer switch

Como se puede ver en la imagen anterior, la condicional del primer switch pregunta por el estado de todos los swiches, si el primer switch está activado (SW = "0000000001") y el estado anterior de los swiches estaban desactivados (and sw1_past = "0000000000"). Entonces se enviará el número '1' al programa de Unity (tx_data_de10 <= "00000001").

4. Paso 4. Se guarda el estado actual de los switches

```
-- Boton1
key1_db_past      <= key1_db;

-- Boton2
key2_db_past      <= key2_db;

-- Sw1 Avansar
sw1_past          <= SW;
sw1_stop_past     <= SW;

-- Sw2 Atras
sw2_past          <= SW;
sw2_stop_past     <= SW;

-- Sw3 Derecha
sw3_past          <= SW;
sw3_stop_past     <= SW;

-- Sw4 Izquierda
sw4_past          <= SW;
sw4_stop_past     <= SW;

END IF;
END PROCESS;
```

Figura 11. Se actualizan las variables auxiliares

Después de que el dato haya sido enviado y antes de que se termine el proceso, se guarda el estado de los switches en una variable para saber cuál es el estado anterior para un futuro ciclo del proceso.

5. Paso 5. Resultado final

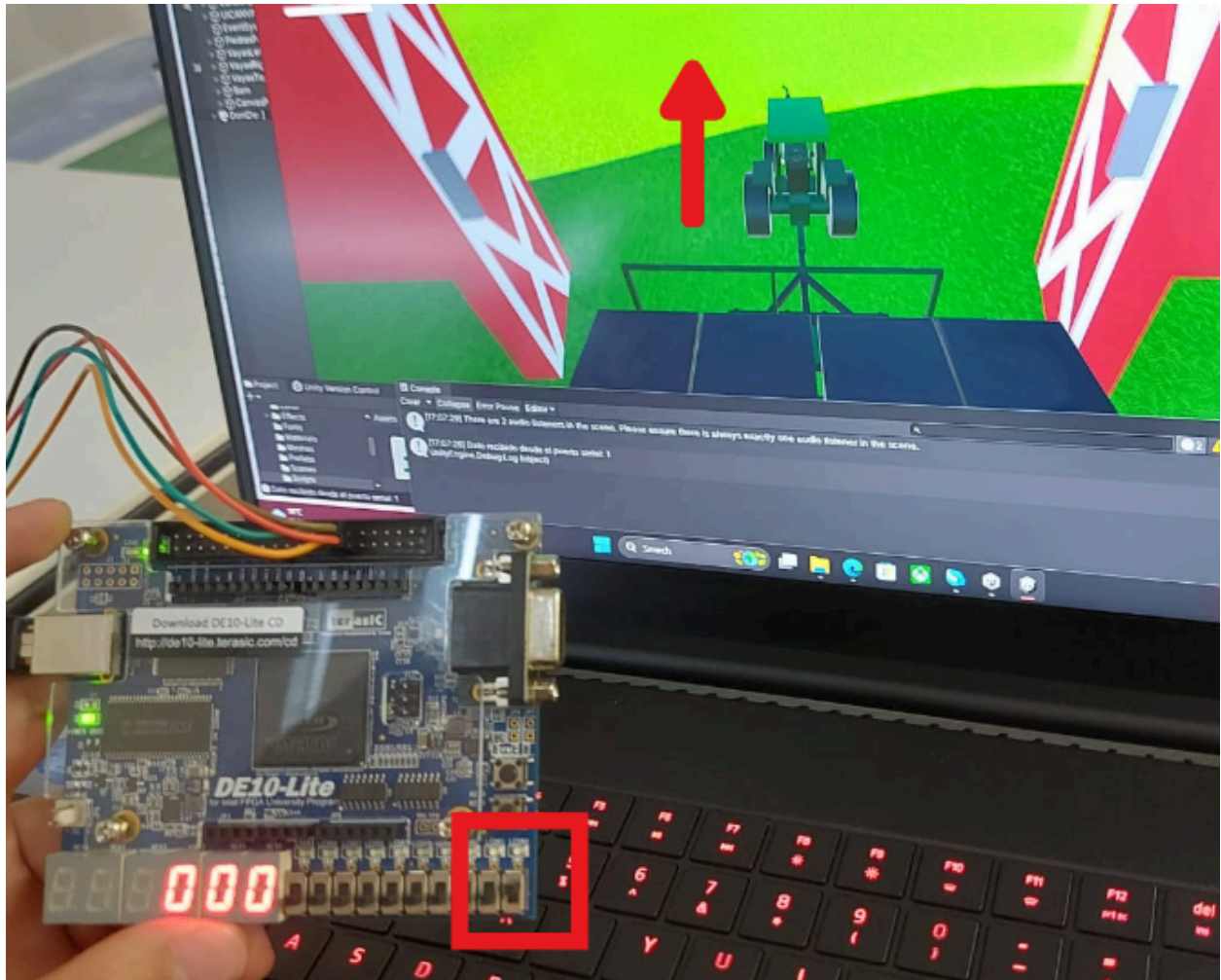


Figura 12. Resultado de activar el primer switch

Como se puede ver en la imagen, al activar el primer switch de la tarjeta FPGA el tractor se mueve hacia adelante.

b) Prueba 2.

- Acción: Aceleración (Botón 1).
- Descripción:
 1. Paso 1: Físicamente se oprime el botón #1 de la tarjeta
 2. Paso 2: Se utilizarán las variables declaradas previo al comienzo la arquitectura

```
-- Señales para componente UART
SIGNAL tx_ena_de10:      std_logic := '1';
SIGNAL tx_data_de10:     std_logic_vector(7 DOWNTO 0);
SIGNAL rx_busy_de10:     std_logic;
SIGNAL rx_error_de10:    std_logic;
SIGNAL rx_data_de10:     std_logic_vector(7 DOWNTO 0); -- Contador de corns
SIGNAL tx_busy_de10:     std_logic;

-- Señales para mandar los datos de la placa
-- Boton 1
SIGNAL key1_db:          std_logic;
SIGNAL key1_db_past:     std_logic := '0';

-- Boton 2
SIGNAL key2_db:          std_logic;
SIGNAL key2_db_past:     std_logic := '0';

-- Sw1 Avansar
SIGNAL sw1_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw1_stop_past:    std_logic_vector(9 DOWNTO 0);

-- Sw2 Atras
SIGNAL sw2_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw2_stop_past:    std_logic_vector(9 DOWNTO 0);

-- Sw3 Derecha
SIGNAL sw3_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw3_stop_past:    std_logic_vector(9 DOWNTO 0);

-- Sw4 Izquierda
SIGNAL sw4_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw4_stop_past:    std_logic_vector(9 DOWNTO 0);
```

Figura 13. Variables para manejar el primer botón

3. Paso 3: Proceso para transmitir datos

```
--Proceso para transmitir datos según las entradas de la interfaz.  
PROCESS( CLOCK_50 )  
BEGIN  
    IF rising_edge( CLOCK_50 ) THEN  
        -- Boton 1  
        IF key1_db = '1' and key1_db_past = '0' THEN  
            tx_ena_de10 <= '0';  
            tx_data_de10 <= "00001001";  
        -- Boton 1  
        ELSIF key1_db = '0' and key1_db_past = '1' THEN  
            tx_ena_de10 <= '0';  
            tx_data_de10 <= "00001010";  
        -- Boton 2  
        ELSIF key2_db = '1' and key2_db_past = '0' THEN  
            tx_ena_de10 <= '0';  
            tx_data_de10 <= "00001011";  
        -- Boton 2  
        ELSIF key2_db = '0' and key2_db_past = '1' THEN  
            tx_ena_de10 <= '0';  
            tx_data_de10 <= "00001100";
```

Figura 14. Condicional para el primer botón

Como se puede ver en la imagen anterior, la condicional del primer botón pregunta por el estado del botón, si el botón 1 está presionado (`key1_db = '1'`) y el estado anterior del botón era no activado (`and key1_db_past = '0'`). Entonces se enviará el número '9' al programa de Unity (`tx_data_de10 <= "00001001"`).

4. Paso 4. Se guarda el estado actual del botón

```
-- Boton1
key1_db_past    <= key1_db;

-- Boton2
key2_db_past    <= key2_db;

-- Sw1 Avansar
sw1_past        <= Sw;
sw1_stop_past   <= Sw;

-- Sw2 Atras
sw2_past        <= Sw;
sw2_stop_past   <= Sw;

-- Sw3 Derecha
sw3_past        <= Sw;
sw3_stop_past   <= Sw;

-- Sw4 Izquirda
sw4_past        <= Sw;
sw4_stop_past   <= Sw;
```

Figura 15. Se actualizan las variables auxiliares

Después de que el dato haya sido enviado y antes de que se termine el proceso, se guarda el estado del botón en una variable para saber cuál es el estado anterior para un futuro ciclo del proceso.

5. Paso 5. Resultado final

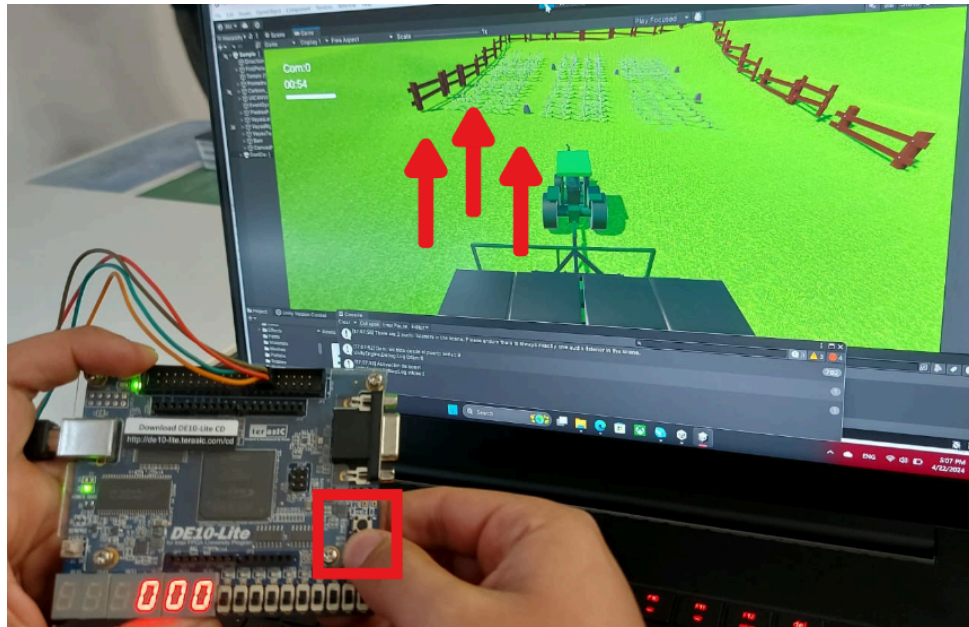


Figura 16. Se actualizan las variables auxiliares

Como se puede ver en la imagen, presionar el botón 1 de la tarjeta FPGA el tractor acelera su velocidad.

c) Prueba 3.

- Acción: Visualización de contador (Display).
- Descripción:

1. Paso 1: En la simulación de Unity se recolecta elementos de cosecha.
2. Paso 2: Se recibe el dato mediante la FPGA y el FTDI en el programa de VHDL.
3. Paso 3: Se inicia el proceso para usar el display de siete segmentos.

```
-- Portmap de componente UART
uart_0 : uart    PORT MAP( CLOCK_50, KEY_TEST, tx_ena_de10, tx_data_de10, GPIO_24, rx_busy_de10, rx_error_de10, rx_data_de10,

-- Portmap para hacer debounce de los botones
button_0 : debounce PORT MAP( CLOCK_50, KEY(1), key1_db );
button_1 : debounce PORT MAP( CLOCK_50, KEY(0), key2_db );

-- Mostrar el contador en las leds [en binario]
LEDR <= "00" & rx_data_de10;

-- Proceso para actualizar el display según los datos recibidos
PROCESS(rx_data_de10)
    variable hundreds : integer range 0 to 9;
    variable tens      : integer range 0 to 9;
    variable single    : integer range 0 to 9;
    variable int_value : integer;

BEGIN

    -- Guardar la variable recibida a binario
    int_value := to_integer(unsigned(rx_data_de10(7 downto 0)));

    -- Extrae centenas, decenas y un dígito.
    hundreds := int_value / 100; -- Hundreds place
    tens      := (int_value / 10) mod 10; -- Tens place
    single    := int_value mod 10; -- Single place
```

Figura 17. Variable de entrada

4. Paso 4: Se transforma el dato recibido a entero y cada dígito del dato se guarda en una variable

```
-- Proceso para actualizar el display según los datos recibidos
PROCESS(rx_data_de10)
    variable hundreds : integer range 0 to 9;
    variable tens      : integer range 0 to 9;
    variable single    : integer range 0 to 9;
    variable int_value : integer;

BEGIN

    -- Guardar la variable recibida a binario
    int_value := to_integer(unsigned(rx_data_de10(7 downto 0)));

    -- Extrae centenas, decenas y un dígito.
    hundreds := int_value / 100; -- Hundreds place
    tens      := (int_value / 10) mod 10; -- Tens place
    single    := int_value mod 10; -- Single place
```

Figura 18. Manejo de dato de entrada

Como se puede observar en la imagen, el dato recibido desde Unity es transformado a un entero mediante la línea [int_value := to_integer(unsigned(rx_data_de10(7 downto 0)))]. El dato es guardado en la variable de "int_value". Nuestro contador llega hasta un máximo de 150. Con lo cual sabemos que el dato recibido será de tres dígitos, estos tres dígitos son guardados en una variable cada uno, se guardan los dígitos de la centena, décima y singular en tres diferentes variables.

5. Paso 5: Visualizar los datos en los segmentos del display

```
-- Asignar valores de visualización basados en dígitos

-- Primer segmento
case single is
  when 0 => Disp_1 <= "1000000"; -- 0
  when 1 => Disp_1 <= "1111001"; -- 1
  when 2 => Disp_1 <= "0100100"; -- 2
  when 3 => Disp_1 <= "0110000"; -- 3
  when 4 => Disp_1 <= "0011001"; -- 4
  when 5 => Disp_1 <= "0010010"; -- 5
  when 6 => Disp_1 <= "0000010"; -- 6
  when 7 => Disp_1 <= "1111000"; -- 7
  when 8 => Disp_1 <= "0000000"; -- 8
  when 9 => Disp_1 <= "0010000"; -- 9
  when others => Disp_1 <= "1000000"; -- 0 by default
end case;

-- Segundo segmento
case tens is
  when 0 => Disp_2 <= "1000000"; -- Blank leading zero
  when 1 => Disp_2 <= "1111001"; -- 1
  when 2 => Disp_2 <= "0100100"; -- 2
  when 3 => Disp_2 <= "0110000"; -- 3
  when 4 => Disp_2 <= "0011001"; -- 4
  when 5 => Disp_2 <= "0010010"; -- 5
  when 6 => Disp_2 <= "0000010"; -- 6
  when 7 => Disp_2 <= "1111000"; -- 7
  when 8 => Disp_2 <= "0000000"; -- 8
  when 9 => Disp_2 <= "0010000"; -- 9
  when others => Disp_2 <= "1000000"; -- 0 by default
end case;

-- Tercer segmento
case hundreds is
  when 0 => Disp_3 <= "1000000"; -- Blank leading zero
  when 1 => Disp_3 <= "1111001"; -- 1
  when 2 => Disp_3 <= "0100100"; -- 2
  when 3 => Disp_3 <= "0110000"; -- 3
  when 4 => Disp_3 <= "0011001"; -- 4
  when 5 => Disp_3 <= "0010010"; -- 5
  when 6 => Disp_3 <= "0000010"; -- 6
  when 7 => Disp_3 <= "1111000"; -- 7
  when 8 => Disp_3 <= "0000000"; -- 8
  when 9 => Disp_3 <= "0010000"; -- 9
  when others => Disp_3 <= "1000000"; -- 0 by default
end case;
```

Figura 19. Implementación de "case"

Como se puede observar en la imagen, para poder ver el resultado del dato en el display, se utiliza 3 “cases” cada uno conlleva una de las variables anteriores. Dependiendo del número cuadrado en la variable es el número que se verá reflejada en el display.

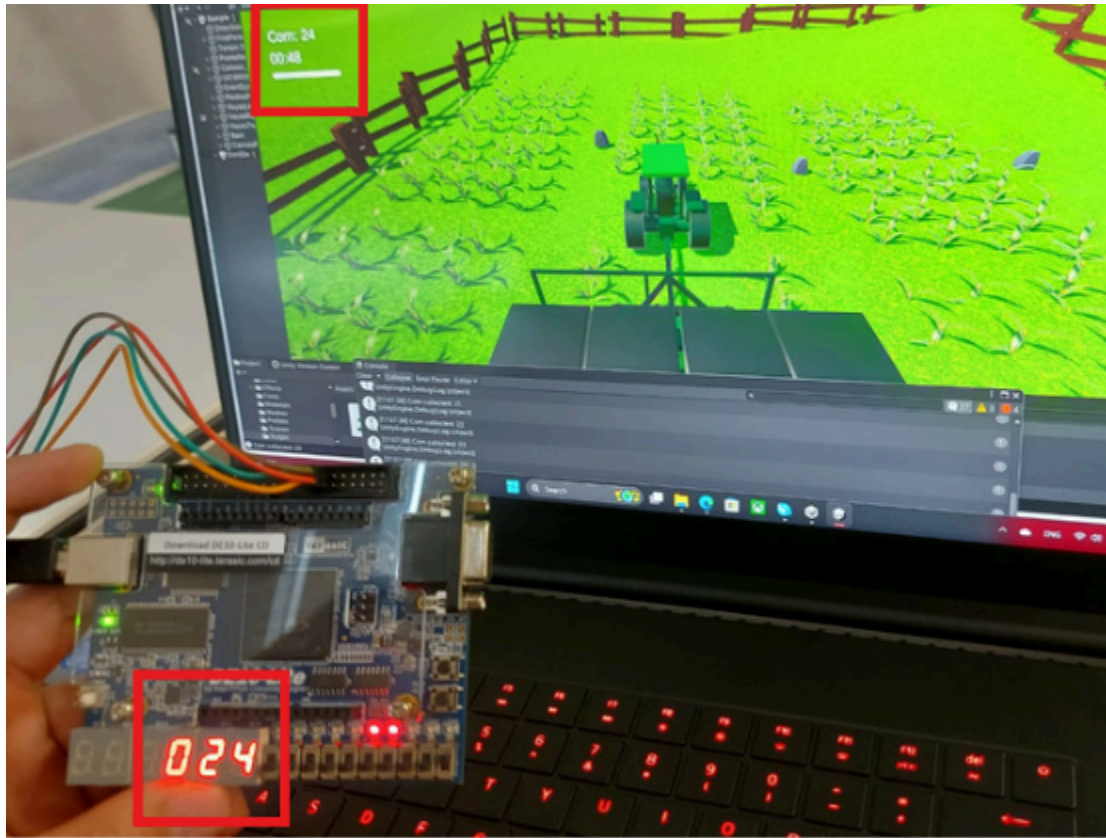


Figura 20. Visualización del display en tiempo real

VIII. Conexión Serial

1. Conexión entre la placa y el juego de Unity

A. Inicialización:

En el método Start(), se configura el Rigidbody del vehículo y se abre el puerto serie para la comunicación. Se establece en qué puerto de entrada se encuentra el FTDI y la su baudrate, También se inicializa una instancia de Corn Collector, que parece en otro script que maneja la recolección de maíz. Así mismo de haber un error al hacer la conexión serial el código enviará el mensaje

```
0 references
void Start()
{
    carRigidbody = gameObject.GetComponent<Rigidbody>();
    carRigidbody.centerOfMass = new Vector3(0, -0.9f, 0);

    serialPort = new SerialPort("COM7", 115200);
    // con esta funcoin al empezar mando a llamar a corn collector
    cornCollector = FindObjectOfType<CornCollector>();
    try
    {
        serialPort.Open();
        serialPort.ReadTimeout = 1;
    }
    catch (System.Exception e)
    {
        Debug.LogError("Error al abrir el puerto serial: " + e.Message);
    }
}
```

Figura 21. Función de start con configuración de Serial Port

B. Actualización por fotograma:

- En el método Update(), se realizan varias acciones:

Se controla el vehículo a través de la función ControlVehicleWithSerial().

Se animan las ruedas del vehículo mediante AnimateWheelMeshes().

Se obtiene la cantidad de maíz recolectado y se envía a través del puerto serie, donde convierte el número del maíz recolectado a bits y manda mediante la conexión serial al FPGA.

```
0 references
void Update()
{
    // Actualizar datos del serial, para hacer uso de los otros puertos
    ControlVehicleWithSerial();
    AnimateWheelMeshes();
    // Obtener la cantidad de maíz recolectado
    int cornCollected = cornCollector.GetCorn();
    // Convertir la cantidad de maíz recolectado a byte
    byte[] cornData = { (byte)cornCollected };
    // Enviar la cantidad de maíz recolectado a través del puerto serie
    SendDataToSerialPort(cornData);
}
```

Figura 22. Función de Update, con inicialización del contador de Corn.

C. Envío de datos al puerto serie:

La función SendDataToSerialPort(byte[] data) se encarga de enviar datos al puerto serie. Si el puerto está abierto, los datos se escriben en él FPGA de lo contrario marca error

```
void SendDataToSerialPort(byte[] data)
{
    if (serialPort != null && serialPort.IsOpen)
    {
        try
        {
            serialPort.Write(data, 0, data.Length);
        }
        catch (System.Exception e)
        {
            Debug.LogWarning("Error al escribir datos en el puerto serial: " + e.Message);
        }
    }
}
```

Figura 23. Función de SendDataToSerialPort con condicionales para envío de datos.

D. Control del vehículo con datos del puerto serie:

La función `ControlVehicleWithSerial()` se encarga de recibir y procesar datos provenientes del puerto serie. Si hay datos disponibles para leer, se leen y se llaman a `ParseSerialData(byte data)` para su interpretación de bits a número.

```
void ControlVehicleWithSerial()
{
    if (serialPort != null && serialPort.IsOpen)
    {
        try
        {
            if (serialPort.BytesToRead > 0)
            {
                byte[] data = new byte[1];
                serialPort.Read(data, 0, 1);
                ParseSerialData(data[0]);
            }
        }
        catch (System.Exception e)
        {
            Debug.LogWarning("Error al leer datos del puerto serial: " + e.Message);
        }
    }
}
```

Figura 24. Función de `ControlVehicleWithSerial`

E. Interpretación de los datos del puerto serie:

ParseSerialData(byte data) interpreta los datos recibidos y toma acciones en consecuencia. Al recibir los bits y convertirlos a números se controlan diferentes acciones del vehículo basadas en los datos recibidos, como girar a la izquierda, girar a la derecha, avanzar, retroceder, activar y desactivar botones de impulso y freno.

```
void ParseSerialData(byte data)
{
    Debug.Log("Dato recibido desde el puerto serial: " + data);

    if (data == 0x01) // Switch 0 arriba
    {
        switch0Activated = true;
        if (!switch1Activated)
        {
            GoForward();
        }
    }
    else if (data == 0x02) // Switch 0 abajo
    {
        switch0Activated = false;
        throttleAxis = 0;
        ApplyMotorTorque(throttleAxis);
    }
    else if (data == 0x03) // Switch 1 arriba
    {
        switch1Activated = true;
        if (!switch0Activated)
        {
            GoReverse();
        }
    }
    else if (data == 0x04) // Switch 1 abajo
    {
        switch1Activated = false;
        throttleAxis = 0;
        ApplyMotorTorque(throttleAxis);
    }
    else if (data == 0x05) // Switch 2 arriba
    {
        switch2Activated = true;
        TurnLeft();
    }
}
```

Figura 25. Función ParseSerialData con las condiciones de los valores de data, para manejo de funciones aplicadas.

F. Control de impulso y freno:

Se han añadido nuevas funciones (boost(), DeactivateBoost(), breaktractor(), DeactivateBreakTractor()) para controlar el impulso y el frenado del vehículo. Estas funciones se activan o desactivan dependiendo de los datos recibidos del puerto serie los cuales son bits también y son interpretados por el puerto serial, si recibe un número habilita el botón y al dejar de presionar el botón recibe otro número.

```
1 reference
public void boost(){
    if(button1Activated){
        // Aca tiene que ir la condiciones para que se realice
        float boostMultiplier = 2.0f; // Esto seria el doble de velocidad
        ApplyMotorTorque(boostMultiplier); //Aca aplicamos el incremento del boostmultiplier
        Debug.Log("Activacion de boost");
    }
}

1 reference
public void DeactivateBoost(){
    ThrottleOff();
}

1 reference
public void breaktractor(){
    if(button2Activated){
        ApplyMotorTorque(0.0f);
        Debug.Log("Freno");
    }
}

1 reference
public void DeactivateBreakTractor(){
    ThrottleOff();
}

1 reference
public void TurnLeft()
{
    if (switch2Activated && !switch3Activated)
    {
        steeringAxis = -1f;
        ApplySteering();
    }
}
```

Figura 26. Declaración de funciones, como boost, break, turnleft y turnright, y su respectiva desactivación

2. Pruebas de funcionamiento

A. Prueba 1: Movimiento del tractor mediante switches

a) Acción: Movimiento a la Izquierda

El FPGA se encuentra conectado a la computadora al igual que el puerto serial (FTDI) en Unity, el script de control del vehículo (PrometeoCarController) tiene dentro de este el funcionamiento de el movimiento a la derecha del tractor.

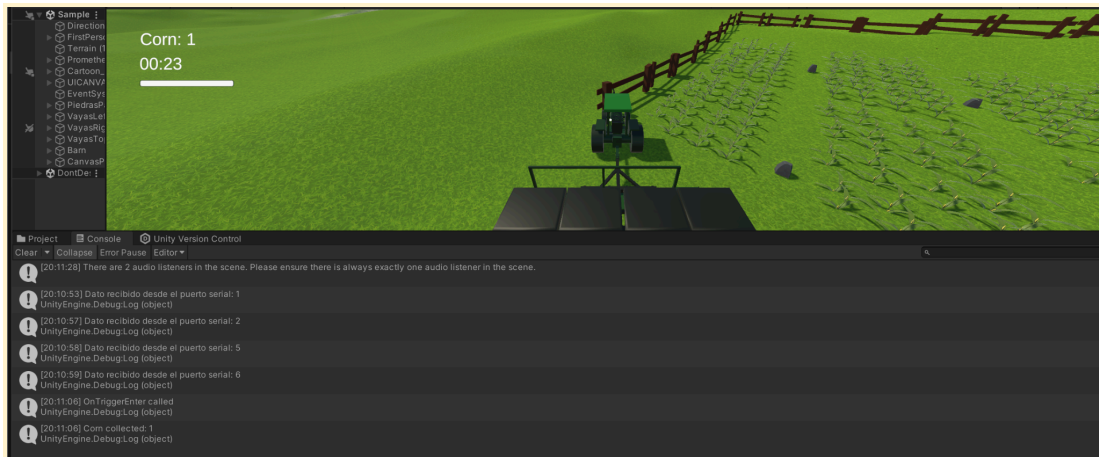


Figura 27. Implementación de envío de recepción de datos a través de la consola de Unity.

b) Descripción

En la FPGA, contiene la lógica necesaria en los 4 switches cada uno da dos valores al subir el switch 0 el valor del bit que arroja es 1 y al bajar el switch es 2, para detectar la activación del switch correspondiente al movimiento a la izquierda y enviar un bit específico a través del puerto serie en este caso para doblar a la izquierda cuando el switch 3 se encuentre arriba en el FPGA el puerto serial envía un 5 el cual en el código es la activación hacia la izquierda y el 6 significa el dejar de realizar esa acción al bajar el switch.

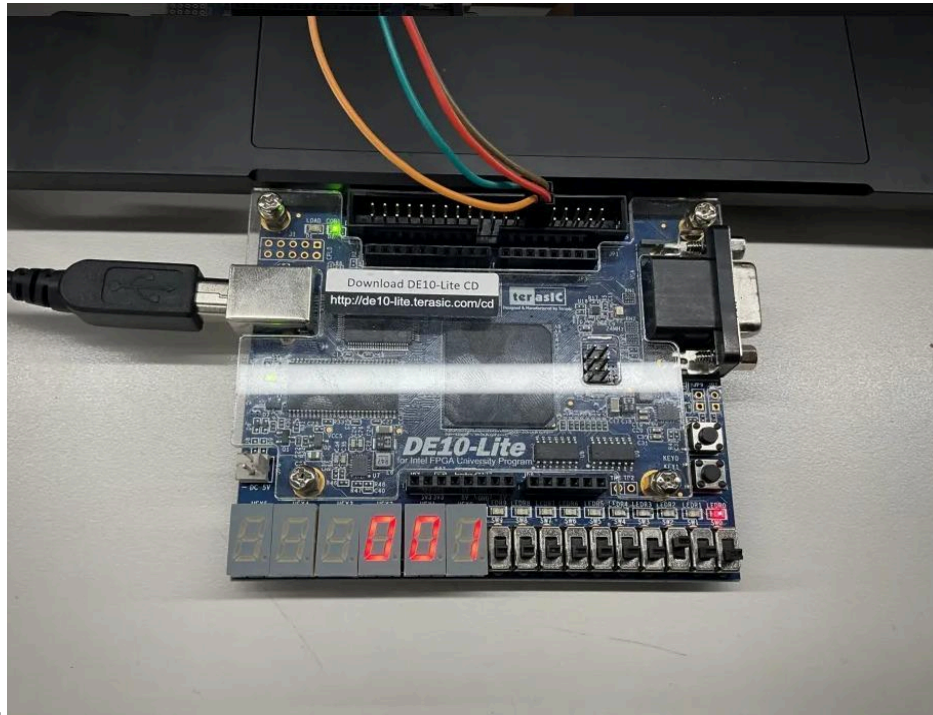


Figura 28. Tercer switch activado

c) Explicaciones de código

En el script `PrometeoCarController` de Unity, implementa la función `ControlVehicleWithSerial()` para recibir y procesar los datos del puerto serie. Agrega una nueva condición en la función `ParseSerialData(byte data)` para interpretar el bit específico enviado desde la FPGA cuando se activa el switch de movimiento a la izquierda.

C/C++

```
else if (data == 0x05) // Switch 3 arriba (movimiento a la izquierda)
{
    switch3Activated = true;
    TurnLeft();
}
else if (data == 0x06) // Switch 3 abajo (detener movimiento a la izquierda)
{
    switch3Activated = false;
    steeringAxis = 0;
    ApplySteering();
}
```

La función `TurnLeft()` en el script `PrometeoCarController` controla el movimiento del tractor hacia la izquierda. La función `TurnLeft()` aplique un ángulo de dirección adecuado a las ruedas del tractor para que gire hacia la izquierda.

C/C++

```
public void TurnLeft()
{
    if (switch3Activated && !switch2Activated)
    {
        steeringAxis = -1f; // Modifica el valor según lo necesario para girar a la izquierda
        ApplySteering();
    }
}
```

B. Prueba 2: Movimiento del tractor mediante botones.

a) Acción: Boost de tractor (más aceleración)

El FPGA se encuentra conectada a la computadora al igual que el puerto serial(FTDI) en Unity, el script de control del vehículo (PrometeoCarController) tiene dentro de este el funcionamiento del boost tractor.

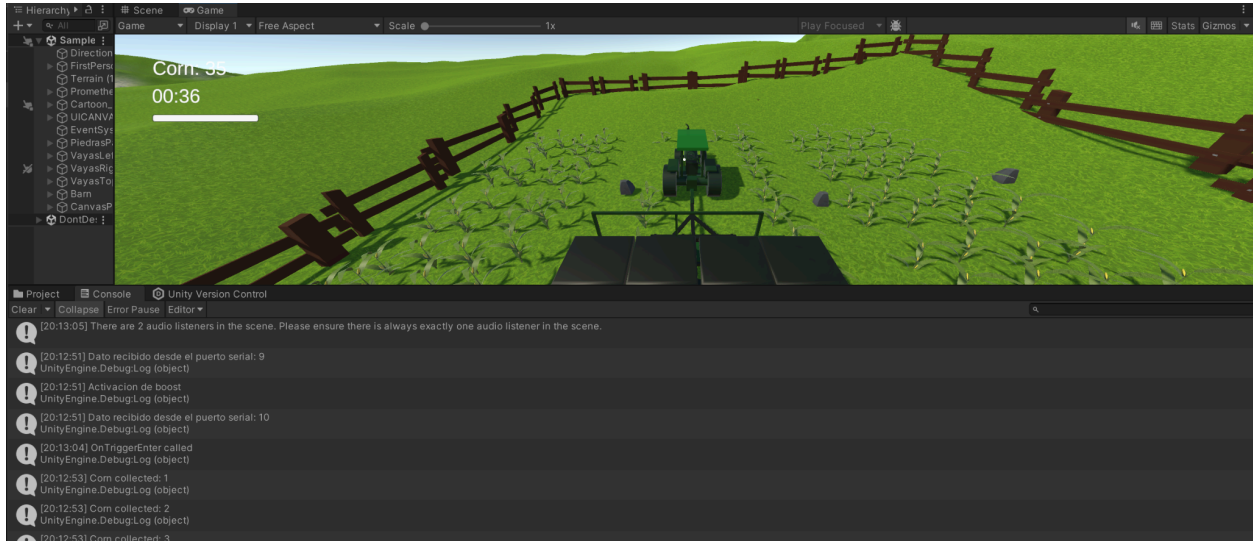


Figura 29. Implementación de los botones y su respectivo valor serial visto desde la consola.

b) Descripción:

En la FPGA, programa la lógica necesaria para detectar la activación del botón correspondiente al movimiento del tractor, al presionar en el FPGA el botón número 0 este envía un 9 y al dejar de presionarlo envía un 10 de esta manera se aplica el boost en el juego.

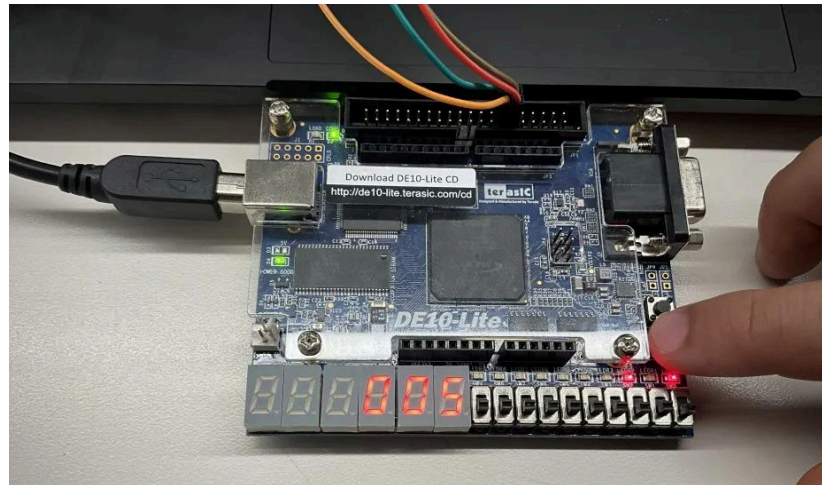


Figura 30. Funcionamiento de los botones, y su respectivo valor serial.

c) Explicaciones de código:

En el script `PrometeoCarController` de Unity, implementa la función `ControlVehicleWithSerial()` para recibir y procesar los datos del puerto serie.

La función `ParseSerialData(byte data)` para interpretar el bit específico enviado desde la FPGA cuando se activa el botón de boost

Al presionar el boton dentro del FPGA se enviara un 9 y al dejar de presionarlo se desactiva la función que tiene el boton

```
C/C++
else if (data == 0x09) // boton 0
{
    button1Activated = true;
    boost();
}
else if (data == 0x0A)
{
    button1Deactivated = false;
    DeactivateBoost();
}
```

C. Prueba 3: Visualización de contador mediante display de FPGA

a) Acción: muestra de datos en Display

El FPGA se encuentra conectada a la computadora al igual que el puerto serial (FTDI) en Unity, el script de control del vehículo (PrometeoCarController) tiene dentro de este el funcionamiento de la conversión de datos de números seriales a bits para el FPGA

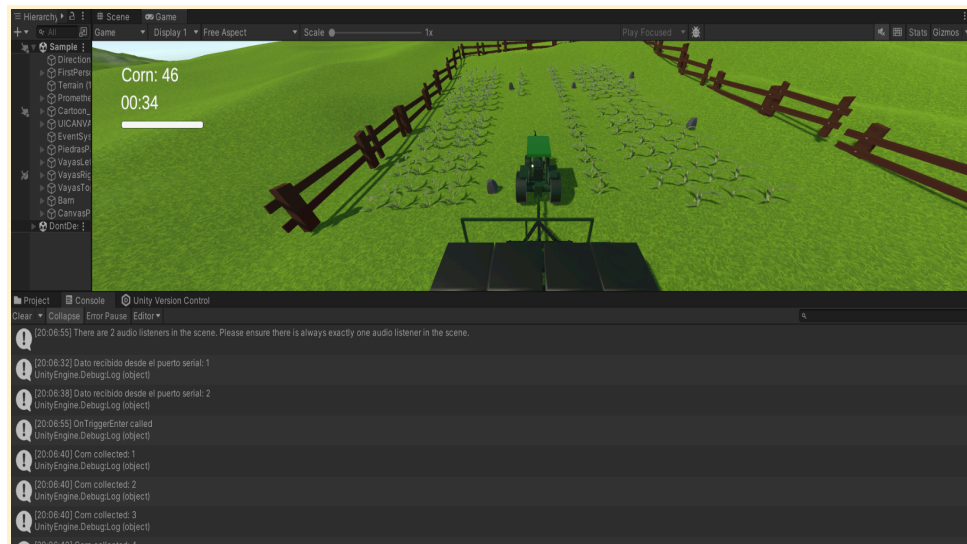


Figura 31. Recepción de datos dentro de la consola.

b) Descripción

Se programó la lógica en la FPGA para que pueda recibir los datos del contador a través del puerto serie.

Configurando el formato de los datos que se enviarán, como bits individuales, estos son enviados desde unity al puerto serial y recibidos en el FPGA.

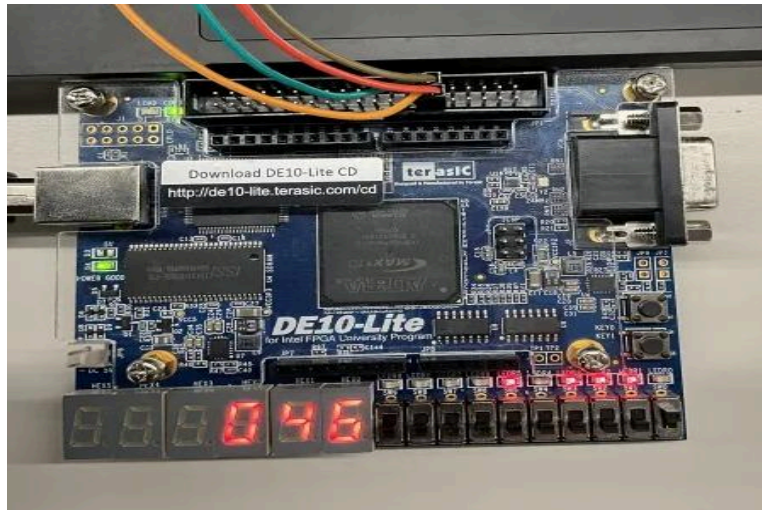


Figura 32. Despliegue del Corn Counter en el FPGA

c) Explicaciones de código

En el código, los datos se envían al display a través del puerto serial utilizando la función `SendDataToSerialPort(byte[] data)`. Esta función realiza los siguientes pasos:

- Verifica si el puerto serial está abierto y disponible para su uso. Si el puerto serial está abierto, utiliza el método `serialPort.Write(data, 0, data.Length)` para escribir los datos en el puerto serial.
- “data” es un arreglo de bytes que contiene los datos que se enviarán al display. El índice 0 indica el primer elemento en el arreglo de bytes desde donde se comenzará a escribir.
- “data.Length” indica la longitud del arreglo de bytes que se va a escribir en el puerto serial.
- Si ocurre algún error durante la escritura de datos en el puerto serial, se registra una advertencia. Dentro de la consola se muestra también la cantidad de maíz recolectado

```
C/C++
void SendDataToSerialPort(byte[] data)
{
    if (serialPort != null && serialPort.IsOpen)
    {
        try
        {
            serialPort.Write(data, 0, data.Length);
        }
        catch (System.Exception e)
        {
            Debug.LogWarning("Error al escribir datos en el puerto serial: " +
e.Message);
        }
    }
}
```

- “`cornCollector.GetCorn()`” obtiene la cantidad de maíz recolectado utilizando el objeto “cornCollector”, que se asume es una instancia de una clase que recolecta maíz (tal vez un objeto de la clase `CornCollector`).
- Esta cantidad de maíz recolectado se convierte en un byte utilizando “`(byte)cornCollected`”.
- El byte resultante se almacena en un arreglo de bytes llamado “cornData.”
- Finalmente, se llama a la función “`SendDataToSerialPort(cornData)`” para enviar el dato del maíz recolectado al display a través del puerto serial.

C/C++

```
void Update()
{
    // Actualizar datos del serial, para hacer uso de los otros puertos
    ControlVehicleWithSerial();
    AnimateWheelMeshes();

    // Obtener la cantidad de maíz recolectado
    int cornCollected = cornCollector.GetCorn();

    // Convertir la cantidad de maíz recolectado a byte
    byte[] cornData = { (byte)cornCollected };

    // Enviar la cantidad de maíz recolectado a través del puerto serie
    SendDataToSerialPort(cornData);
}
```

IX. Video Demostrativo Etapa 2

[Etapa 2 Equipo 6 Video Hardware \(youtube.com\)](#)

X. Prototipo Final

1. Funcionamiento

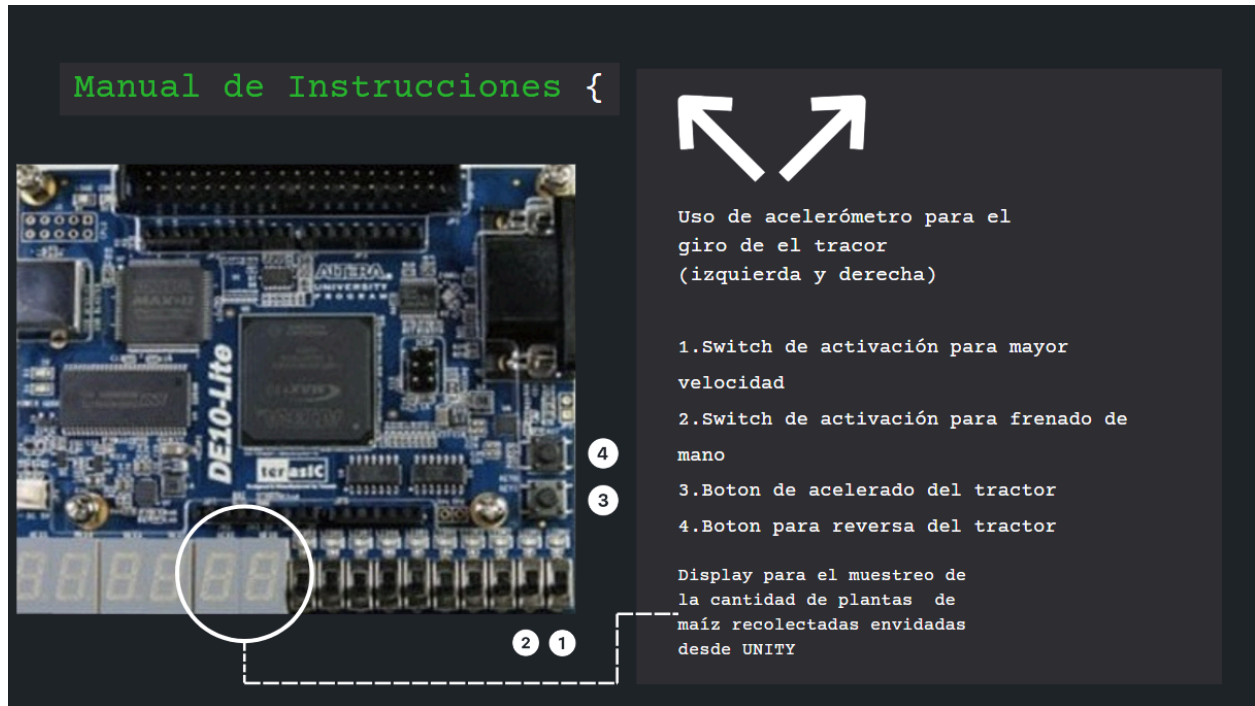


Figura 33. Funcionamiento de la FPGA

2. Acelerómetro y Procesador

- Gumnut VHDI

Código completo en sección de anexo (Anexo 3.A)

```
314
315
316  -----
317  -- Gumnut code
318  -----
319
320  -- Output => TX_START -> data memory address: 0x00
321  PROCESS( CLOCK_50, rst_i )
322  BEGIN
323      IF rst_i = '1' THEN
324          tx_ena_de10 <= '1';
325      ELSIF rising_edge( CLOCK_50 ) THEN
326          IF port_adr_o = "00000000" and -- address port
327             port_cyc_o = '1' and -- control signals for I/O
328             port_stb_o = '1' and
329             port_we_o = '1' -- 'write' operation
330          THEN
331              tx_ena_de10 <= port_dat_o(0);
332          END IF;
333      END IF;
334  END PROCESS;
335
336
337
338  -- Input <= KEY1 -> data memory address: 0x03
339  PROCESS( CLOCK_50, rst_i )
340  BEGIN
341      IF rst_i = '1' THEN
342          key1_domy <= ( OTHERS => '0' );
343      ELSIF rising_edge( CLOCK_50 ) THEN
344
345          -- Key1 (key1)
346          IF port_adr_o = "00000011" and -- address port
347             port_cyc_o = '1' and -- control signals for I/O
348             port_stb_o = '1' and
349             port_we_o = '0' -- 'read' operation
350          THEN
351              key1_domy <= "0000000" & key1_db;
352          END IF;
353      END IF;
354  END PROCESS;
355
356
357
```

Figura 34. Proceso TX_Start y Botón 1

Como se puede observar en la imagen, se puede ver dos procesos para la comunicación de Gumnut. El primero proceso es para la variable de TX_Start, ya que es una variable que recibimos desde Assembly, se requiere tener el puerto “port_we_o” igual a ‘1’. El valor recibido es enviado a la variable de Uart “tx_ena_de10”. El segundo proceso es tiene el mismo diseño que todos los procesos para los inputs de la FPGA. Se tiene el proceso con lista de sensibilidad del reloj y el botón de reset, se tiene un “if” para verificar que el reset esté en 0, y posterior de eso tenemos un “if” para verificar que las variables de Uart estén con el valor que deseamos, donde “port_cyc_o” y “port_stb_o” son igual a 1 y “port_we_o” es igual a 0. Lo único que cambia en los procesos es la variable de “port_adr_o” en este caso el primero botón en el

listado de data en Assembly está en el espacio 3, con lo cual igualamos esta variable a "11". Por último, enviamos la variable del primer botón al Assembly.

```

359
360 -- Input <= KEY0 -> data memory address: 0x04
361 PROCESS( CLOCK_50, rst_i )
362 BEGIN
363     IF rst_i = '1' THEN
364         key2_domy <= ( OTHERS => '0' );
365     ELSIF rising_edge( CLOCK_50 ) THEN
366
367         -- Key1 (key1)
368         IF port_adr_o = "00000100" and -- address port
369            port_cyc_o = '1' and -- control signals for I/O
370            port_stb_o = '1' and
371            port_we_o = '0' -- 'read' operation
372         THEN
373             key2_domy <= "0000000" & key2_db;
374         END IF;
375     END IF;
376
377 END PROCESS;
378
379
380 -- Input <= SW -> data memory address: 0x05
381 PROCESS( CLOCK_50, rst_i )
382 BEGIN
383     IF rst_i = '1' THEN
384         sw_domy <= ( OTHERS => '0' );
385     ELSIF rising_edge( CLOCK_50 ) THEN
386
387         -- Key1 (key1)
388         IF port_adr_o = "00000101" and -- address port
389            port_cyc_o = '1' and -- control signals for I/O
390            port_stb_o = '1' and
391            port_we_o = '0' -- 'read' operation
392         THEN
393             sw_domy <= Sw(7 downto 0);
394         END IF;
395     END IF;
396
397 END PROCESS;
398
399
400

```

Figura 35. Proceso botón 2 y swicthes

Del mismo al proceso del botón 1 se realizó los procesos del botón 2 y los swicthes, para el botón 2 el adress cambia a 4 en binario y para los swicthes cambia a 5 en binario. Esto se debe a sus posiciones en el listado de data en Assembly. Se envía el dato del botón o de los siwcthes a sus variables correspondientes.

```

401 -- Input <= ACCE -> data memory address: 0x06
402 PROCESS( CLOCK_50, rst_i )
403 BEGIN
404     IF rst_i = '1' THEN
405         acce_domy <= ( OTHERS => '0' );
406     ELSIF rising_edge( CLOCK_50 ) THEN
407
408         -- Key1 (key1)
409         IF port_adr_o = "00000110" and -- address port
410            port_cyc_o = '1' and -- control signals for I/O
411            port_stb_o = '1' and
412            port_we_o = '0' -- 'read' operation
413         THEN
414             acce_domy <= LEDR(8 downto 1);
415         END IF;
416     END IF;
417 END PROCESS;
418
419
420
421
422
423
424 -- Output => TX_DATA -> data memory address: 0x02
425 PROCESS( CLOCK_50, rst_i )
426 BEGIN
427     IF rst_i = '1' THEN
428         tx_data_de10 <= ( OTHERS => '0' );
429     ELSIF rising_edge( CLOCK_50 ) THEN
430         IF port_adr_o = "00000010" and -- address port
431            port_cyc_o = '1' and -- control signals for I/O
432            port_stb_o = '1' and
433            port_we_o = '1' -- 'write' operation
434         THEN
435             tx_data_de10 <= port_dat_o;
436         END IF;
437     END IF;
438 END PROCESS;
439
440
441 -- Select procedure to send the correct input data to Gumnut
442 WITH port_adr_o SELECT
443
444     port_dat_i <= key1_domy WHEN "00000011",
445                  key2_domy WHEN "00000100",
446                  sw_domy  WHEN "00000101",
447                  acce_domy WHEN "00000110",
448                  UNAFFECTED WHEN OTHERS;
449
450 END Structural;

```

Figura 36. Proceso acelerómetro, TX_Data y Select

De igual manera se realizó el proceso para el acelerómetro, donde se envía la variable de LEDR, cuál es nuestro valor del acelerómetro, sin enviar los extremos, ya que la variable debe de ser de 8 bits y LEDR es de 10 bits. Por último tenemos el proceso para TX_Data en donde recibimos el número clave procesado en Assembly, ya que este proceso es de input, se requiere de poner el "port_we_o" igual a 1. Finalmente, tenemos un proceso de SELECT, que nos permite enviar los valores correctos a "port_dat_i" dependiendo de que address se esté manejando.

- Gumnut Assembly

```
1      text
2      org 0x000 ; start here on reset
3      jmp main
4      ; Data memory layout
5      data
6      TX_START:  bss 1      ;0
7      tx_disable: byte 1      ;1
8      TX_DATA:   bss 1      ;2
9      KEY1:      bss 1      ;3
10     KEY2:      bss 1      ;4
11     SW:        bss 1      ;5
12     ACCE:      bss 1      ;6
13     KEY1_ON:   byte 1      ;key1 on    = 1
14     KEY1_OFF:  byte 2      ;key1 off   = 2
15     KEY2_ON:   byte 3      ;key2 on    = 3
16     KEY2_OFF:  byte 4      ;key2 off   = 4
17     SW1_ON:    byte 9      ;sw1 on     = 9
18     SW1_OFF:   byte 10     ;sw1 off    = 10
19     SW2_ON:    byte 11     ;sw2 on     = 11
20     SW2_OFF:   byte 12     ;sw2 off    = 12
21     ACCE_OFF1: byte 6      ;acce off   = 6
22     ACCE_OFF2: byte 8      ;acce off   = 8
23     ACCE_R:    byte 7      ;acce right = 7
24     ACCE_L:    byte 5      ;acce left  = 5
25
```

Figura 37. Etiquetas principales

Este fragmento de código en assembly define la disposición de la memoria de datos para un programa que utiliza Gumnut. Establece ubicaciones de memoria para variables como tx_disable (que controla la transmisión), KEY1 y KEY2 (relacionadas con los botones), SW (asociada con los swiches) y ACCE (que almacena el estado del acelerómetro). Además, define constantes como KEY1_ON, KEY1_OFF, etc., para representar los numero claves que serán enviados a Unity, dependiendo del estado de las variables de la FPGA.

```

26 ; Main program
27     text
28     org 0x010
29
30 main:    ldm r1, r0           ;r1 = 0
31         inp r1, KEY1         ;r1 = key1
32         jsb delay            ;delay
33         and r1, r1, 1        ;r1 = r1 and 1
34         bz not_key1         ;if r1 = 0 jump to not_key1
35         ldm r1, r0           ;if !r1 = 0 then r1 = 0
36         ldm r2, KEY1_ON      ;r2 = key1 on
37         out r2, TX_DATA      ;Data = r2
38         out r0, TX_START     ;Start = 0
39         ldm r2, tx_disable   ;r2 = 1
40         out r2, TX_START     ;Start = 1
41         jmp next_sw1        ;jumps
42
43
44 ;when key1 is not pressed
45 not_key1: ldm r1, r0         ;r1 = 0
46         ldm r2, KEY1_OFF     ;r2 = key1 off
47         out r2, TX_DATA      ;Data = r2
48         out r0, TX_START     ;Start = 0
49         ldm r2, tx_disable   ;r2 = 1
50         out r2, TX_START     ;start = 1
51         jmp next_sw1        ;jumps
52

```

Figura 38. Proceso para checar por el botón 1

Este fragmento de código verifica el estado del primero botón y manda un '1' cuando es presionando y un '2' cuando no.

```

53 ;check for swicth 1
54 next_sw1:    ldm r1, r0           ;r1 = 0
55              inp r1, SW           ;r1 = swicthes
56              jsb delay            ;dealy
57              and r1, r1, 1        ;r1 = r1 and 1
58              bz not_sw1           ;if r1 = 0
59              ldm r1, r0           ;if r1 = 1
60              ldm r2, SW1_ON       ;r2 = sw1 on
61              out r2, TX_DATA      ;Data = r2
62              out r0, TX_START     ;start = 0
63              ldm r2, tx_disable   ;r2 = 1
64              out r2, TX_START     ;start = 1
65              jmp next_sw2         ;jumps
66
67 ;when sw1 is off
68 not_sw1:     ldm r1, r0           ;r1 = 0
69              ldm r2, SW1_OFF      ;r2 = sw1 off
70              out r2, TX_DATA      ;Data = r2
71              out r0, TX_START     ;start = 0
72              ldm r2, tx_disable   ;r2 = 1
73              out r2, TX_START     ;strat = 1
74              jmp next_sw2         ;jumps
75

```

Figura 39. Proceso para checar por el switch 1

Este fragmento de código verifica el estado del primer switch, manda un 9 cuando está activado y un 10 cuando no

```

76 ;check for switch 2
77 next_sw2:    ldm r1, r0                ;r1 = 0
78              inp r1, SW                ;r1 = swicthes
79              jsb delay                 ;dealy
80              and r1, r1, 2             ;r1 = r1 and 2
81              bz not_sw2                ;if r1 = 0
82              ldm r1, r0                ;if r1 = 1
83              ldm r2, SW2_ON            ;r2 = sw2 on
84              out r2, TX_DATA            ;data = r2
85              out r0, TX_START           ;start = 0
86              ldm r2, tx_disable        ;r2 = 1
87              out r2, TX_START           ;start = 1
88              jmp next_key2             ;jumps
89
90 ;when sw2 is not on
91 not_sw2:     ldm r1, r0                ;r1 = 0
92              ldm r2, SW2_OFF           ;r2 = sw2 off
93              out r2, TX_DATA            ;data = r2
94              out r0, TX_START           ;start = 0
95              ldm r2, tx_disable        ;r2 = 1
96              out r2, TX_START           ;start = 1
97              jmp next_key2             ;jumps
98

```

Figura 40. Proceso para checar por el switch 2

Este fragmento de código verifica el estatus del segundo switch, manda un 11 cuando está activado y un 12 cuando no.

```

99      ;check for key2
100     next_key2:  ldm r1, r0           ;r1 = 0
101                  inp r1, KEY2        ;r1 = key2
102                  jsb delay            ;delay
103                  and r1, r1, 1        ;r1 = r1 and 1
104                  bz not_key2          ;r1 = 0
105                  ldm r1, r0           ;if r1 = 1
106                  ldm r2, KEY2_ON      ;r2 = key2 on
107                  out r2, TX_DATA      ;data = r2
108                  out r0, TX_START     ;start = 0
109                  ldm r2, tx_disable   ;r2 = 1
110                  out r2, TX_START     ;start = 1
111                  jmp next_acce        ;jumps
112
113     ;when key 2 is not pressed
114     not_key2:    ldm r1, r0           ;r1 = 0
115                  ldm r2, KEY2_OFF      ;r2 = key2 off
116                  out r2, TX_DATA      ;data = r2
117                  out r0, TX_START     ;start = 0
118                  ldm r2, tx_disable   ;r2 = 1
119                  out r2, TX_START     ;start = 1
120                  jmp next_acce        ;jumps
121

```

Figura 41. Proceso para checar por el botón 2

Este fragmento de código verifica el estatus del segundo botón, envía un 3 cuando está activado y un 4 cuando no.

```

122 ;check for accelerometer
123 next_acce: ldm r1, r0           ;r1 = 0
124             inp r1, ACCE        ;r1 = acce
125             jsb delay           ;delay
126             and r3, r1, 24      ;r3 = r1 and 00011000
127             bnz not_acce       ;if acce = middle
128
129             and r3, r1, 1       ;r3 = r1 and 00000001
130             bnz acce_right      ;if acce = right
131             and r3, r1, 2       ;r3 = r1 and 00000010
132             bnz acce_right      ;if acce = right
133             and r3, r1, 4       ;r3 = r1 and 00000100
134             bnz acce_right      ;if acce = right
135
136
137             and r3, r1, 32      ;r3 = r1 and 00100000
138             bnz acce_left       ;if acce = left
139             and r3, r1, 64      ;r3 = r1 and 01000000
140             bnz acce_left       ;if acce = left
141             and r3, r1, 128     ;r3 = r1 and 10000000
142             bnz acce_left       ;if acce = left
143
144             ldm r1, r0           ;r1 = 0
145             ldm r2, ACCE_OFF     ;r2 = acce off
146             out r2, TX_DATA      ;data = r2
147             out r0, TX_START     ;start = 0
148             ldm r2, tx_disable   ;r2 = 1
149             out r2, TX_START     ;start = 1
150             jsb delay           ;delay
151             jmp main             ;loops back to main
152

```

Figura 42. Proceso para checar por el acelerómetro 1/2

Este fragmento de código verifica el estatus de acelerómetro y salta a su etiqueta correspondiente dependiendo si la tarjeta está en el medio, a la derecha o a la izquierda.

```

153 ;when acce is in the middle
154 not_acce:   ldm r1, r0           ;r1 = 0
155             ldm r2, ACCE_OFF1   ;r2 = acce off 1
156             out r2, TX_DATA      ;data = r2
157             out r0, TX_START     ;start = 0
158             ldm r2, tx_disable   ;r2 = 1
159             out r2, TX_START     ;start = 1
160             jsb delay            ;delay
161             ldm r1, r0           ;r1 = 0
162             ldm r2, ACCE_OFF2   ;r2 = acce off 2
163             out r2, TX_DATA      ;data = r2
164             out r0, TX_START     ;start = 0
165             ldm r2, tx_disable   ;r2 = 1
166             out r2, TX_START     ;start = 1
167             jsb delay            ;delay
168             jmp main            ;loops back to main
169
170 ;when acce is tilt right
171 acce_right: ldm r1, r0           ;r1 = r0
172             ldm r2, ACCE_R       ;r2 = acce right
173             out r2, TX_DATA      ;data = r2
174             out r0, TX_START     ;start = 0
175             ldm r2, tx_disable   ;r2 = 1
176             out r2, TX_START     ;start = 1
177             jsb delay            ;delay
178             jmp main            ;loops back to main
179
180 ;when acce is tilt left
181 acce_left:  ldm r1, r0           ;r1 = r0
182             ldm r2, ACCE_L       ;r2 = acce left
183             out r2, TX_DATA      ;data = r2
184             out r0, TX_START     ;start = 0
185             ldm r2, tx_disable   ;r2 = 1
186             out r2, TX_START     ;start = 1
187             jsb delay            ; delay
188             jmp main            ;loops back to main
189

```

Figura 43. Proceso para checar por el acelerómetro 2/2

Etiquetas para el acelerómetro manda en número 7 cuando está en la derecha, el número 5 cuando está en la izquierda y el número 6 y 8 cuando está en el medio.

```

190 ;delay
191 delay:      add r7, r0, 0
192 again3:    add r6, r0, 0
193 again2:    add r5, r0, 0
194 again1:    add r5, r5, 1
195           sub r0, r5, 0xFF
196           bnz again1
197           add r6, r6, 1
198           sub r0, r6, 0xFF
199           bnz again2
200           add r7, r7, 1
201           sub r0, r7, 0x0C
202           bnz again3
203           ret

```

Figura 44. Proceso del delay

Por último tenemos el delay que se usa frecuentemente para procesar los datos de forma correcta.

3. Diseño RTL

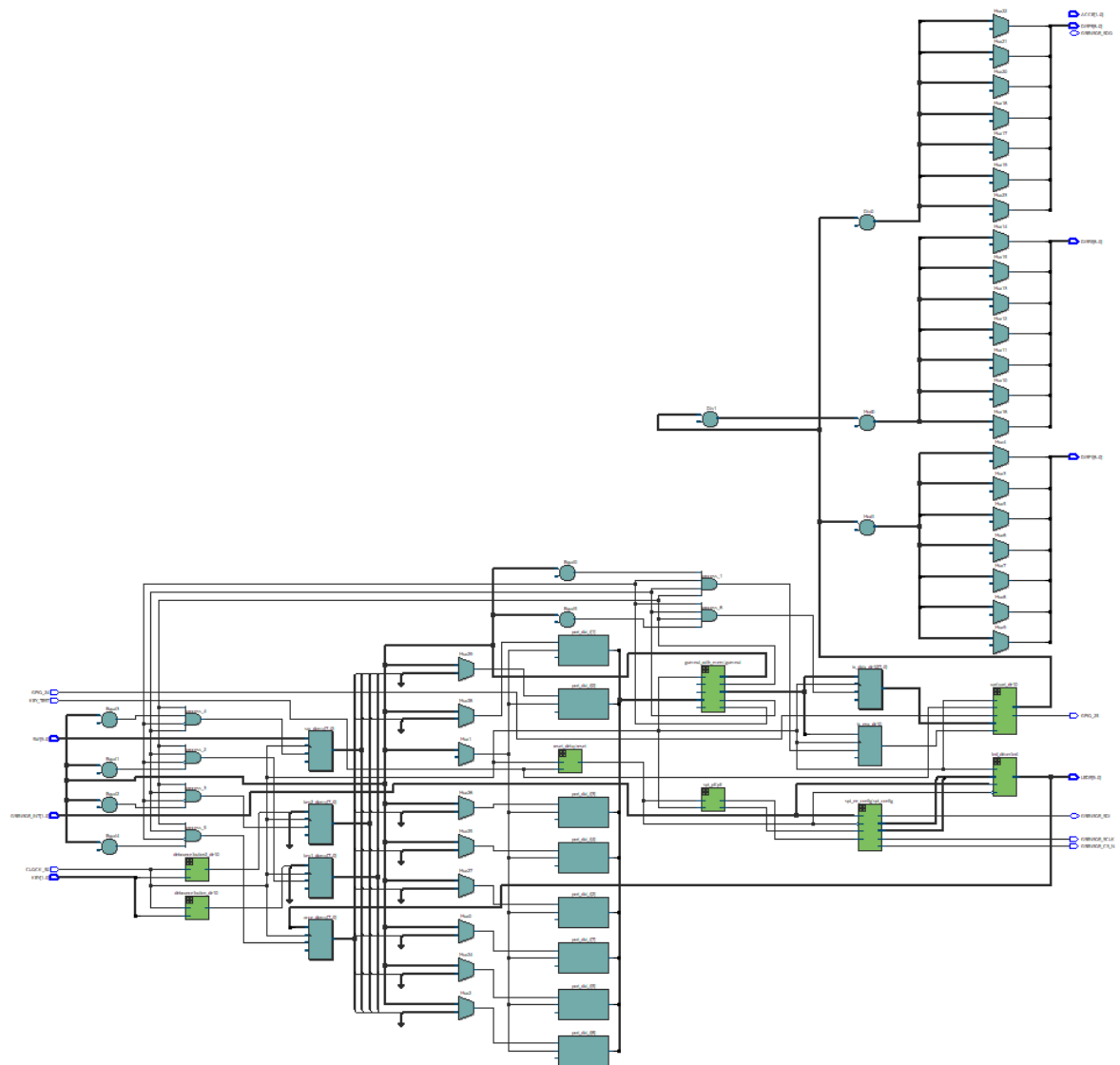


Figura 45. Diseño RTL completo

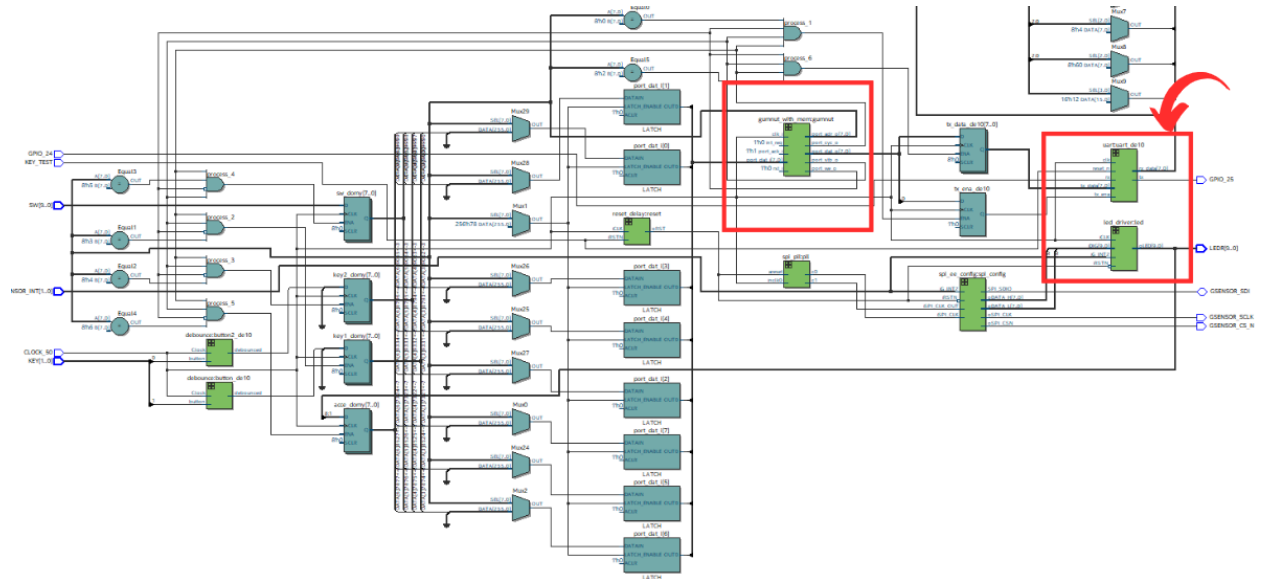


Figura 46. Sección más importante del diseño RTL

4. Video demostrativo prototipo final

https://drive.google.com/file/d/1qZjMsV-cHfJ-5JC24LBhDQOBC66vc_vc/view?usp=sharing

5. Conclusiones

Rosendo: La combinación de un FPGA de 10 Lite y Unity para controlar un tractor en simulación ofrece eficiencia y flexibilidad. El FPGA permite implementar lógica personalizada de manera eficiente, mientras que Unity ofrece un entorno visual para pruebas exhaustivas. La reconfigurabilidad del FPGA agiliza el desarrollo y reduce costos. Esta convergencia tecnológica fomenta la colaboración entre disciplinas, como ingeniería de hardware y software. En conjunto, permiten soluciones sólidas y atractivas. La integración ofrece un ejemplo emocionante de cómo la tecnología puede resolver desafíos complejos de manera eficiente y efectiva en entornos de simulación, personalmente considero que el poder trabajar de manera equitativa con mi equipo fue una experiencia muy buena y el poder colaborar juntos para poder llegar a la solución planteada .

Hugo: La convergencia de tecnologías como Unity y un FPGA 10 Lite para el control de simuladores, como en el caso del tractor, subraya el poder transformador de la colaboración entre disciplinas diversas. Esta sinergia no solo impulsa la innovación en ingeniería, sino que también resalta la importancia de la adaptabilidad y la creatividad en un mundo tecnológicamente cambiante, reforzando así la necesidad de abrazar la diversidad como motor de la innovación y el progreso. Esto pues nos resulta factible para hacer simulaciones de diferentes procesos, y verificarlos antes de ponerlos a prueba, para evitar errores y costos de producción.

Jesús: Durante el proyecto abordé, desde la implementación de máquinas de estado hasta la integración de Unity en el control de simuladores, ha sido una experiencia reveladora. A través de esta experiencia, he comprendido la importancia de la colaboración entre disciplinas en el desarrollo de soluciones tecnológicas avanzadas. La combinación de un FPGA de 10 Lite y Unity ha demostrado ser una estrategia eficaz para el control de simuladores, ofreciendo eficiencia, flexibilidad y reconfigurabilidad. Esta convergencia tecnológica no solo ha permitido la creación de soluciones sólidas y atractivas, sino que también ha fomentado un ambiente de trabajo colaborativo y productivo.

Roberto: La integración entre un FPGA 10 Lite y Unity para controlar simulaciones, como el tractor, demuestra el valor de unir disciplinas tecnológicas distintas. Este enfoque facilita la implementación de lógica personalizada con eficiencia a través del FPGA, mientras que Unity proporciona un entorno visual para pruebas detalladas. La flexibilidad y reconfigurabilidad del FPGA aceleran el desarrollo y reducen costos. Esta colaboración tecnológica destaca cómo la diversidad y la colaboración entre ingeniería de hardware y software pueden impulsar soluciones sólidas y atractivas. Este tipo de sinergia destaca la importancia de adaptarse y ser creativo en un entorno tecnológico en constante evolución, promoviendo la innovación y el progreso mediante la capacidad de simular y verificar procesos antes de implementarlos, lo cual ayuda a evitar errores y costos innecesarios.

XI. Bibliografía

ZyBooks. TE 2002B: Design with Programmable Logic home. 2024

Deere & Company. (n.d.). John Deere Innovating since 1837. John Deere History | Tractor History | John Deere US. <https://www.deere.com/en/our-company/history/>

Technologies, T. (n.d.). All FPGA boards - max 10 - DE10-lite board. <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=1021>

Unity Learn. (n.d.). Learn game development w/ unity: Courses & tutorials in game design, VR, AR, & real-time 3D. <https://learn.unity.com/tutorials?signup=true>

LuisCanary. (2022, 28 marzo). *Como SALIR de tu Videojuego en Unity*  (EXIT Button) [Vídeo].

YouTube. <https://www.youtube.com/watch?v=1-CYqMMUvbk>

Coco Code. (2020, 10 diciembre). *Create great GAME OVER screen in Unity UI - Unity tutorial* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=K4uOjb5p3Io>

BravePixelG. (2022, 23 septiembre). *Cómo habilitar el siguiente nivel cuando eliminamos a los enemigos de la escena en Unity* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=uBnSbCSJ-aY>

SpeedTutor. (2018, 18 julio). *EMPTY GAMEOBJECTS in ONE CLICK - Unity* [Vídeo]. YouTube. https://www.youtube.com/watch?v=sCdGQOO1_JU

Dev Ritch. (2023, 6 agosto). *Como hacer un Menú de pausa en Unity || Tutorial 2023* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=wbBo08iTrQU>

John Deere España. (2021, 5 julio). *X9 - la cosechadora de 100 toneladas | John Deere ES* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=0nZLPnQaTic>

Multiverso Sensorial. (2021, 9 agosto). *Vida y Daño en Unity* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=rTuWlvZ8TYI>

Rigor Mortis Tortoise. (2023, 25 febrero). *How to Collect Coins in Unity 3D Tutorial 2023 (Updated)* [Vídeo]. YouTube. https://www.youtube.com/watch?v=6iSJ_jh6Rdo

XII. Anexos

1. Tutoriales de Unity completados

A. Tutorial 1

<https://youtu.be/udLSkhMP1Gw>

B. Tutorial 2

https://youtu.be/iP5M7ApWB_4

C. Tutorial 3

<https://youtu.be/YpdBQ9hasyl>

2. Codigos Etapa #2

A. Codigos de Placa VHDL

- Codigo Interfaz

Python

```
-- Equipo#6 Situacion Problema Etapa #2. Avance de Funcionalidad
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

-- Entidad
ENTITY de10_lite IS
    PORT(
        CLOCK_50 : IN      std_logic;           -- Reloj
        KEY       : IN      std_logic_vector(1 DOWNTO 0); -- Botones
        SW        : IN      std_logic_vector(9 DOWNTO 0); -- Switches
        KEY_TEST  : IN      std_logic;           -- Variable auxiliar para llamar al objeto Uart
        GPIO_24   : IN      std_logic;           -- RX
        GPIO_25   : OUT std_logic;               -- TX
        LEDR      : OUT std_logic_vector(9 DOWNTO 0); -- Leds
        Disp_1    : OUT std_logic_vector(6 downto 0); -- First segment of the display
        Disp_2    : OUT std_logic_vector(6 downto 0); -- Second segment of the display
        Disp_3    : OUT std_logic_vector(6 downto 0); -- Third segment of the display
    );
END;

-- Arquitectura
ARCHITECTURE Structural OF de10_lite IS

    -- Inicializacion de componente UART
    COMPONENT uart IS
        GENERIC(
            clk_freq :integer := 50_000_000; --frequency of system clock in Hertz
            baud_rate :integer := 115_200;   --data link baud rate in bits/second
            os_rate   :integer := 16;        --oversampling rate to find center of receive bits (in samples per baud period)
            d_width   :integer := 8;         --data bus width
            parity    :integer := 0;         --0 for no parity, 1 for parity
            parity_eo :std_logic:= '0';      --'0' for even, '1' for odd parity
        )
        PORT(
            clk      : IN      std_logic;           --system clock
            reset_n  : IN      std_logic;           --asynchronous reset
            tx_ena   : IN      std_logic;           --initiate transmission
            tx_data  : IN      std_logic_vector(d_width-1 DOWNTO 0); --data to transmit
            rx       : IN      std_logic;           --receive pin
            rx_busy  : OUT std_logic;               --data reception in progress, LEDR(9)
            rx_error : OUT std_logic;               --start, parity, or stop bit error detected
            rx_data  : OUT std_logic_vector(d_width-1 DOWNTO 0); --data received
            tx_busy  : OUT std_logic;               --transmission in progress, LEDR(8)
            tx       : OUT std_logic;               --transmit pin
        );
    END COMPONENT;

    -- Inicializacion de componente Debounce
```

```
COMPONENT debounce IS
```

```
    PORT (
        Clock      :      IN      STD_LOGIC;
        button     :      IN      STD_LOGIC;
        debounced  :      BUFFER  STD_LOGIC);
```

```
END COMPONENT;
```

```
-- Señales para componente UART
```

```
SIGNAL tx_ena_de10:      std_logic := '1';
SIGNAL tx_data_de10:     std_logic_vector(7 DOWNTO 0);
SIGNAL rx_busy_de10:     std_logic;
SIGNAL rx_error_de10:    std_logic;
SIGNAL rx_data_de10:     std_logic_vector(7 DOWNTO 0); -- Contador de corns
SIGNAL tx_busy_de10:     std_logic;
```

```
-- Señales para mandar los datos de la placa
```

```
-- Boton 1
SIGNAL key1_db:          std_logic;
SIGNAL key1_db_past:     std_logic := '0';
```

```
-- Boton 2
SIGNAL key2_db:          std_logic;
SIGNAL key2_db_past:     std_logic := '0';
```

```
-- SW1 Avansar
SIGNAL sw1_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw1_stop_past:    std_logic_vector(9 DOWNTO 0);
```

```
-- SW2 Atras
SIGNAL sw2_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw2_stop_past:    std_logic_vector(9 DOWNTO 0);
```

```
-- SW3 Derecha
SIGNAL sw3_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw3_stop_past:    std_logic_vector(9 DOWNTO 0);
```

```
-- SW4 Izquierda
SIGNAL sw4_past:         std_logic_vector(9 DOWNTO 0);
SIGNAL sw4_stop_past:    std_logic_vector(9 DOWNTO 0);
```

```
BEGIN
```

```
-- Portmap de componente UART
uart_0 : uart      PORT MAP( CLOCK_50, KEY_TEST, tx_ena_de10, tx_data_de10, GPIO_24, rx_busy_de10,
rx_error_de10, rx_data_de10, tx_busy_de10, GPIO_25 );
```

```
-- Portmap para hacer debounce de los botones
button_0 : debounce      PORT MAP( CLOCK_50, KEY(1), key1_db );
button_1 : debounce      PORT MAP( CLOCK_50, KEY(0), key2_db );
```

```
-- Mostrar el contador en las leds [en binario]
LEDR <= "00" & rx_data_de10;
```

```
-- Proceso para actualizar el display según los datos recibidos
```

```
PROCESS(rx_data_de10)
    variable hundreds : integer range 0 to 9;
    variable tens      : integer range 0 to 9;
    variable single    : integer range 0 to 9;
    variable int_value : integer;
```

```
BEGIN
```

```

-- Guardar la variable recibida a binario
int_value := to_integer(unsigned(rx_data_de10(7 downto 0)));

-- Extrae centenas, decenas y un dígito.
hundreds := int_value / 100;           -- Hundreds place
tens      := (int_value / 10) mod 10;   -- Tens place
single    := int_value mod 10;         -- Single place

-- Asignar valores de visualización basados en dígitos

-- Primer segmento
case single is
    when 0 => Disp_1 <= "1000000"; -- 0
    when 1 => Disp_1 <= "1111001"; -- 1
    when 2 => Disp_1 <= "0100100"; -- 2
    when 3 => Disp_1 <= "0110000"; -- 3
    when 4 => Disp_1 <= "0011001"; -- 4
    when 5 => Disp_1 <= "0010010"; -- 5
    when 6 => Disp_1 <= "0000010"; -- 6
    when 7 => Disp_1 <= "1111000"; -- 7
    when 8 => Disp_1 <= "0000000"; -- 8
    when 9 => Disp_1 <= "0010000"; -- 9
    when others => Disp_1 <= "1000000"; -- 0 by default
end case;

-- Segundo segmento
case tens is
    when 0 => Disp_2 <= "1000000"; -- Blank leading zero
    when 1 => Disp_2 <= "1111001"; -- 1
    when 2 => Disp_2 <= "0100100"; -- 2
    when 3 => Disp_2 <= "0110000"; -- 3
    when 4 => Disp_2 <= "0011001"; -- 4
    when 5 => Disp_2 <= "0010010"; -- 5
    when 6 => Disp_2 <= "0000010"; -- 6
    when 7 => Disp_2 <= "1111000"; -- 7
    when 8 => Disp_2 <= "0000000"; -- 8
    when 9 => Disp_2 <= "0010000"; -- 9
    when others => Disp_2 <= "1000000"; -- 0 by default
end case;

-- Tercer segmento
case hundreds is
    when 0 => Disp_3 <= "1000000"; -- Blank leading zero
    when 1 => Disp_3 <= "1111001"; -- 1
    when 2 => Disp_3 <= "0100100"; -- 2
    when 3 => Disp_3 <= "0110000"; -- 3
    when 4 => Disp_3 <= "0011001"; -- 4
    when 5 => Disp_3 <= "0010010"; -- 5
    when 6 => Disp_3 <= "0000010"; -- 6
    when 7 => Disp_3 <= "1111000"; -- 7
    when 8 => Disp_3 <= "0000000"; -- 8
    when 9 => Disp_3 <= "0010000"; -- 9
    when others => Disp_3 <= "1000000"; -- 0 by default
end case;

END PROCESS;

--Proceso para transmitir datos según las entradas de la interfaz.
PROCESS( CLOCK_50 )
BEGIN
    IF rising_edge( CLOCK_50 ) THEN

        -- Boton 1

```

```

IF key1_db = '1' and key1_db_past = '0' THEN
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001001";

-- Boton 1
ELSIF key1_db = '0' and key1_db_past = '1' THEN
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001010";

-- Boton 2
ELSIF key2_db = '1' and key2_db_past = '0' THEN
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001011";

-- Boton 2
ELSIF key2_db = '0' and key2_db_past = '1' THEN
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001100";

-- Avansar
-- SW = 1
ELSIF(SW = "000000001" and sw1_past = "000000000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000001";

-- SW = 0
ELSIF(SW = "000000000" and sw1_stop_past = "000000001") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000010";

-- Atras
-- SW = 2
ELSIF(SW = "000000010" and sw2_past = "000000000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000011";

-- SW = 0
ELSIF(SW = "000000000" and sw2_stop_past = "000000010") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000100";

-- Derecha
-- SW = 3
ELSIF(SW = "000000100" and sw3_past = "000000000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000101";

-- SW = 0
ELSIF(SW = "000000000" and sw3_stop_past = "000000100") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000110";

-- Izquierda
-- SW = 4
ELSIF(SW = "000001000" and sw4_past = "000000000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00000111";

-- SW = 0
ELSIF(SW = "000000000" and sw4_stop_past = "000001000") then
    tx_ena_de10 <= '0';
    tx_data_de10 <= "00001000";

-- Esle
ELSE
    tx_ena_de10 <= '1';
    tx_data_de10 <= "00000000";

END IF;

```

```

-- Boton1
key1_db_past      <= key1_db;

-- Boton2
key2_db_past      <= key2_db;

-- SW1 Avansar
sw1_past          <= SW;
sw1_stop_past     <= SW;

-- SW2 Atras
sw2_past          <= SW;
sw2_stop_past     <= SW;

-- SW3 Derecha
sw3_past          <= SW;
sw3_stop_past     <= SW;

-- SW4 Izquierda
sw4_past          <= SW;
sw4_stop_past     <= SW;

END IF;
END PROCESS;

END Structural1;

```

- Codigo UART

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;
```

```

Entidad
ENTITY uart IS
  GENERIC(

    clk_freq  : INTEGER  := 50_000_000; --frequency of system clock in Hertz
    baud_rate : INTEGER  := 115_200;    --data link baud rate in bits/second
    os_rate   : INTEGER  := 16;         --oversampling rate to find center of receive bits (in samples per baud period)
    d_width   : INTEGER  := 8;          --data bus width
    parity    : INTEGER  := 0;          --0 for no parity, 1 for parity
    parity_eo : STD_LOGIC:= '0');       --'0' for even, '1' for odd parity

  PORT(

    clk      : IN    STD_LOGIC;          --system clock
    reset_n  : IN    STD_LOGIC;          --asynchronous reset
    tx_ena   : IN    STD_LOGIC;          --initiate transmission
    tx_data  : IN    STD_LOGIC_VECTOR(d_width-1 DOWNTO 0); --data to transmit
    rx       : IN    STD_LOGIC;          --receive pin
    rx_busy  : OUT   STD_LOGIC;          --data reception in progress
    rx_error : OUT   STD_LOGIC;          --start, parity, or stop bit error detected
    rx_data  : OUT   STD_LOGIC_VECTOR(d_width-1 DOWNTO 0); --data received
    tx_busy  : OUT   STD_LOGIC;          --transmission in progress
    tx       : OUT   STD_LOGIC;          --transmit pin

  );

END uart;

-- Arquitectura
ARCHITECTURE logic OF uart IS
  TYPE tx_machine IS(idle, transmit); --transmit state machine data type
  TYPE rx_machine IS(idle, receive);  --receive state machine data type
  SIGNAL tx_state : tx_machine;        --transmit state machine
  SIGNAL rx_state : rx_machine;        --receive state machine
  SIGNAL baud_pulse : STD_LOGIC := '0'; --periodic pulse that occurs at the baud rate
  SIGNAL os_pulse : STD_LOGIC := '0';  --periodic pulse that occurs at the oversampling rate
  SIGNAL parity_error : STD_LOGIC;      --receive parity error flag
  SIGNAL rx_parity : STD_LOGIC_VECTOR(d_width DOWNTO 0); --calculation of receive parity
  SIGNAL tx_parity : STD_LOGIC_VECTOR(d_width DOWNTO 0); --calculation of transmit parity
  SIGNAL rx_buffer : STD_LOGIC_VECTOR(parity+d_width DOWNTO 0) := (OTHERS => '0'); --values received
  SIGNAL tx_buffer : STD_LOGIC_VECTOR(parity+d_width+1 DOWNTO 0) := (OTHERS => '1'); --values to be transmitted
BEGIN

  --generate clock enable pulses at the baud rate and the oversampling rate
  PROCESS(reset_n, clk)
    VARIABLE count_baud : INTEGER RANGE 0 TO clk_freq/baud_rate-1 := 0; --counter to determine baud rate period
    VARIABLE count_os : INTEGER RANGE 0 TO clk_freq/baud_rate/os_rate-1 := 0; --counter to determine oversampling period
  BEGIN

    IF(reset_n = '0') THEN --asynchronous reset asserted
      baud_pulse <= '0'; --reset baud rate pulse
      os_pulse <= '0'; --reset oversampling rate pulse
      count_baud := 0; --reset baud period counter
      count_os := 0; --reset oversampling period counter
    ELSE
      ELIF(clk'EVENT AND clk = '1') THEN
        --create baud enable pulse
        IF(count_baud < clk_freq/baud_rate-1) THEN --baud period not reached
          count_baud := count_baud + 1; --increment baud period counter
          baud_pulse <= '0'; --deassert baud rate pulse
        ELSE
          --baud period reached
        END IF
      END IF
    END IF
  END PROCESS
END logic;

```

```

        count_baud := 0;                                --reset baud period counter
        baud_pulse <= '1';                               --assert baud rate pulse
        count_os := 0;                                   --reset oversampling period counter to avoid cumulative error
    END IF;

    --create oversampling enable pulse
    IF(count_os < clk_freq/baud_rate/os_rate-1) THEN    --oversampling period not reached
        count_os := count_os + 1;                      --increment oversampling period counter
        os_pulse <= '0';                               --deassert oversampling rate pulse
    ELSE                                                --oversampling period reached
        count_os := 0;                                  --reset oversampling period counter
        os_pulse <= '1';                               --assert oversampling pulse
    END IF;
END IF;
END IF;
END PROCESS;

--receive state machine
PROCESS(reset_n, clk)

    VARIABLE rx_count : INTEGER RANGE 0 TO parity+d_width+2 := 0;    --count the bits received
    VARIABLE os_count : INTEGER RANGE 0 TO os_rate-1 := 0;          --count the oversampling rate pulses

BEGIN
    IF(reset_n = '0') THEN                                         --asynchronous reset asserted
        os_count := 0;                                             --clear oversampling pulse counter
        rx_count := 0;                                             --clear receive bit counter
        rx_busy <= '0';                                           --clear receive busy signal
        rx_error <= '0';                                           --clear receive errors
        rx_data <= (OTHERS => '0');                                --clear received data output
        rx_state <= idle;                                          --put in idle state
    ELSIF(clk'EVENT AND clk = '1' AND os_pulse = '1') THEN      --enable clock at oversampling rate

        CASE rx_state IS

            WHEN idle =>                                           --idle state
                rx_busy <= '0';                                     --clear receive busy flag

                IF(rx = '0') THEN                                  --start bit might be present

                    IF(os_count < os_rate/2) THEN                --oversampling pulse counter is not at start bit center
                        os_count := os_count + 1;                --increment oversampling pulse counter
                        rx_state <= idle;                          --remain in idle state

                    ELSE                                           --oversampling pulse counter is at bit center
                        os_count := 0;                             --clear oversampling pulse counter
                        rx_count := 0;                             --clear the bits received counter
                        rx_busy <= '1';                             --assert busy flag
                        rx_buffer <= rx & rx_buffer(parity+d_width DOWNT0 1); --shift the start bit into receive buffer
                        rx_state <= receive;                       --advance to receive state

                    END IF;

                ELSE                                              --start bit not present
                    os_count := 0;                                 --clear oversampling pulse counter
                    rx_state <= idle;                              --remain in idle state

                END IF;

            WHEN receive =>                                       --receive state

                IF(os_count < os_rate-1) THEN                    --not center of bit
                    os_count := os_count + 1;                    --increment oversampling pulse counter
                    rx_state <= receive;                          --remain in receive state

                ELSIF(rx_count < parity+d_width) THEN            --center of bit and not all bits received

```

```

        os_count := 0;                                --reset oversampling pulse counter
        rx_count := rx_count + 1;                      --increment number of bits received counter
        rx_buffer <= rx & rx_buffer(parity+d_width DOWNT0 1); --shift new received bit into receive buffer
        rx_state <= receive;                           --remain in receive state

    ELSE
        rx_data <= rx_buffer(d_width DOWNT0 1);        --center of stop bit
        rx_error <= rx_buffer(0) OR parity_error OR NOT rx; --output data received to user logic
        rx_busy <= '0';                                --output start, parity, and stop bit error flag
        rx_state <= idle;                              --deassert received busy flag
                                                --return to idle state

    END IF;

END CASE;

END IF;

END PROCESS;

--receive parity calculation logic
rx_parity(0) <= parity_eo;
rx_parity_logic: FOR i IN 0 TO d_width-1 GENERATE
    rx_parity(i+1) <= rx_parity(i) XOR rx_buffer(i+1);
END GENERATE;

WITH parity SELECT --compare calculated parity bit with received parity bit to determine error
    parity_error <= rx_parity(d_width) XOR rx_buffer(parity+d_width) WHEN 1, --using parity
    '0' WHEN OTHERS; --not using parity

--transmit state machine
PROCESS(reset_n, clk)

    VARIABLE tx_count : INTEGER RANGE 0 TO parity+d_width+3 := 0; --count bits transmitted

BEGIN

    IF(reset_n = '0') THEN
        tx_count := 0; --asynchronous reset asserted
        tx <= '1'; --clear transmit bit counter
        tx_busy <= '1'; --set tx pin to idle value of high
        tx_state <= idle; --set transmit busy signal to indicate unavailable
        tx_state <= idle; --set tx state machine to ready state

    ELSIF(clk'EVENT AND clk = '1') THEN

        CASE tx_state IS

            WHEN idle => --idle state

                IF(tx_ena = '0') THEN --new transaction latched in
                    tx_buffer(d_width+1 DOWNT0 0) <= tx_data & '0' & '1'; --latch in data for transmission and start/stop bits

                    IF(parity = 1) THEN --if parity is used
                        tx_buffer(parity+d_width+1) <= tx_parity(d_width); --latch in parity bit from parity logic
                    END IF;

                    tx_busy <= '1'; --assert transmit busy flag
                    tx_count := 0; --clear transmit bit count
                    tx_state <= transmit; --proceed to transmit state

                ELSE --no new transaction initiated
                    tx_busy <= '0'; --clear transmit busy flag
                    tx_state <= idle; --remain in idle state
                END IF;

            END IF;

```

```

        WHEN transmit =>                                --transmit state

        IF(baud_pulse = '1') THEN                        --beginning of bit
            tx_count := tx_count + 1;                    --increment transmit bit counter
            tx_buffer <= '1' & tx_buffer(parity+d_width+1 DOWNT0 1);--shift transmit buffer to output next bit

        END IF;

        IF(tx_count < parity+d_width+3) THEN              --not all bits transmitted
            tx_state <= transmit;                        --remain in transmit state

        ELSE                                              --all bits transmitted

            tx_state <= idle;                             --return to idle state

        END IF;

    END CASE;
    tx <= tx_buffer(0);                                --output last bit in transmit transaction buffer

END IF;

END PROCESS;

--transmit parity calculation logic
tx_parity(0) <= parity_eo;
tx_parity_logic: FOR i IN 0 to d_width-1 GENERATE
    tx_parity(i+1) <= tx_parity(i) XOR tx_data(i);

END GENERATE;

END logic;

```

- Codigo debounce

Python

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all;
-- This module can be used to debounce a KEY on the DE10-Lite board

-- Entidad
ENTITY debounce IS
    PORT (
        Clock      :      IN      STD_LOGIC;
        button      :      IN      STD_LOGIC;
        debounced   :      BUFFER  STD_LOGIC);

END debounce;

-- Arquitectura
ARCHITECTURE Behavior OF debounce IS

    SIGNAL COUNT : STD_LOGIC_VECTOR(2 DOWNTO 0);
    SIGNAL done, counting : STD_LOGIC;

BEGIN

    PROCESS
    BEGIN

        WAIT UNTIL Clock'EVENT AND Clock = '1';

        IF (done = '1' AND button = '1') THEN -- When counter expires and button is no longer
            debounced <= '0';                -- pressed, then set the debounced output to 0

        ELSIF (button = '0') THEN             -- When the "button" is pressed
            debounced <= '1';                -- set the debounced output to 1

        END IF;

    END PROCESS;

    PROCESS
    BEGIN

        WAIT UNTIL Clock'EVENT AND Clock = '1';

        IF (done = '1') THEN
            Count <= "000";

        ELSIF (debounced = '1') THEN
            Count <= Count + '1';

        END IF;

    END PROCESS;

    done <= '1' WHEN Count = "111" ELSE '0';

END Behavior;
```

3. Códigos Prototipo Final

A. Código VHDL (Entidad principal)

Python

```
-- Program for the simulation of a tractor using the elements of FPGA and the Gumnut process
-- Equipo#6

LIBRARY ieee;
USE ieee.std_logic_1164.all, ieee.numeric_std.all;

ENTITY de10_lite IS
    PORT(
        -- FPGA elementos
        CLOCK_50          : IN          std_logic;
        -- Clock
        KEY                : IN          std_logic_vector( 1 DOWNTO 0 );
        -- Botons
        SW                 : IN          std_logic_vector( 9 DOWNTO 0 );
        -- Swiches
        LEDR               : BUFFER std_logic_vector( 9 DOWNTO 0 );
        Leds
        DISP1              : OUT         std_logic_vector(6 downto 0);
        First segment of the display
        DISP2              : OUT         std_logic_vector(6 downto 0);
        Second segment of the display
        DISP3              : OUT         std_logic_vector(6 downto 0);
        Third segment of the display
        ACCE               : BUFFER std_logic_vector(1 downto 0);
        segment of the display
        -- Uart signals
        GPIO_24            : IN          std_logic;
        --RX
        GPIO_25            : OUT std_logic;
        KEY_TEST           : IN          std_logic;
        -- Auxiliary variable to call the Uart object

        -- Accelerometer signals
        GSENSOR_INT        : IN          std_logic_vector(1 DOWNTO 0);
        GSENSOR_SDI        : INOUT      std_logic;
        GSENSOR_SDO        : INOUT      std_logic;
        GSENSOR_CS_N       : OUT         std_logic;
        GSENSOR_SCLK       : OUT         std_logic
    );
END de10_lite;

ARCHITECTURE Structural OF de10_lite IS
    -----
    -- Gumnut Component
    -----

    component gumnut_with_mem IS
        generic(
            IMem_file_name      : string := "gasm_text.dat";
            DMem_file_name      : string := "gasm_data.dat";
            debug                : boolean := false
        );
        port(
            clk_i               : in      std_logic;

            rst_i               : in      std_logic;
            -- I/O port bus
            port_cyc_o          : out     std_logic;
            port_stb_o          : out     std_logic;
            port_we_o           : out     std_logic;
            port_ack_i          : in      std_logic;
        );
    end component;
end Structural;
```

```

port_adr_o          : out unsigned(7 downto 0);
port_dat_o          : out std_logic_vector(7 downto 0);
port_dat_i          : in      std_logic_vector(7 downto 0);
-- Interrupts
int_req             : in std_logic;
int_ack             : out std_logic
    );
end component gumnut_with_mem;

-- Gumnut signals
SIGNAL clk_i, rst_i          : std_logic;
SIGNAL port_cyc_o, port_stb_o : std_logic;
SIGNAL port_we_o, port_ack_i  : std_logic;
SIGNAL int_req, int_ack       : std_logic;
SIGNAL port_adr_o            : unsigned(7 DOWNTO 0);
SIGNAL port_dat_o, port_dat_i : std_logic_vector(7 DOWNTO 0);

-----
-- Uart Components
-----

COMPONENT uart IS
    GENERIC(
        clk_freq      : integer := 50_000_000; --frequency of system clock in Hertz
        baud_rate      : integer := 115_200;   --data link baud rate in bits/second
        os_rate        : integer := 16;        --oversampling rate to find center of
receive bits (in samples per baud period)
        d_width        : integer := 8;         --data bus width
        parity          : integer := 0;         --0 for no parity, 1 for parity
        parity_eo : std_logic := '0'           --'0' for even, '1' for odd parity
    );
    PORT(
        clk      : IN std_logic;                --system clock
        reset_n   : IN std_logic;                --asynchronous reset
        tx_ena    : IN std_logic;                --initiate transmission
        tx_data   : IN std_logic_vector(d_width-1 DOWNTO 0); --data to transmit
        rx        : IN std_logic;                --receive pin
        rx_busy   : OUT std_logic;               --data reception in progress, LEDR(9)
        rx_error  : OUT std_logic;               --start, parity, or stop bit error
        detected

        rx_data   : OUT std_logic_vector(d_width-1 DOWNTO 0); --data received
        tx_busy   : OUT std_logic;               --transmission in progress, LEDR(8)
        tx        : OUT std_logic
        --transmit pin
    );
END COMPONENT;

COMPONENT debounce IS
    PORT(
        Clock      : IN std_logic;
        button     : IN std_logic;
        debounced : BUFFER std_logic
    );
END COMPONENT;

-- Uart signals
SIGNAL tx_ena_de10 : std_logic := '1';
SIGNAL tx_data_de10, rx_data_de10 : std_logic_vector(7 DOWNTO 0);
SIGNAL rx_busy_de10, rx_error_de10 : std_logic;
SIGNAL tx_busy_de10 : std_logic;

SIGNAL key1_db : std_logic;
SIGNAL key2_db : std_logic;

-----
-- Accelerometer Components

```

```

-----

COMPONENT reset_delay IS
    PORT(
        iRSTN    : IN std_logic;
        iCLK     : IN std_logic;
        oRST     : OUT  std_logic
    );
END COMPONENT;

COMPONENT spi_pll IS
    PORT(
        areset   : IN std_logic;
        inclk0   : IN std_logic;
        c0       : OUT  std_logic;
        c1       : OUT  std_logic
    );
END COMPONENT;

COMPONENT spi_ee_config IS
    PORT(
        iRSTN    : IN std_logic;
        iSPI_CLK : IN std_logic;
        iSPI_CLK_OUT : IN  std_logic;
        iG_INT2   : IN std_logic;
        oDATA_L   : OUT std_logic_vector(7 DOWNTO 0);
        oDATA_H   : OUT std_logic_vector(7 DOWNTO 0);
        SPI_SDIO  : INOUT std_logic;
        oSPI_CSN  : OUT std_logic;
        oSPI_CLK  : OUT std_logic
    );
END COMPONENT;

COMPONENT led_driver IS
    PORT(
        iRSTN    : IN std_logic;
        iCLK     : IN std_logic;
        iDIG     : IN std_logic_vector(9 DOWNTO 0);
        iG_INT2   : IN std_logic;
        oLED     : OUT std_logic_vector(9 DOWNTO 0)
    );
END COMPONENT;

-- Accelerometer signals
SIGNAL dly_rst: std_logic;
SIGNAL spi_clk: std_logic;
SIGNAL spi_clk_out: std_logic;
SIGNAL data_x:  std_logic_vector(15 DOWNTO 0);

SIGNAL sw_domy:          std_logic_vector(7 downto 0);
SIGNAL key1_domy:        std_logic_vector(7 downto 0);
SIGNAL key2_domy:        std_logic_vector(7 downto 0);
SIGNAL acce_domy:        std_logic_vector(7 downto 0);

BEGIN

-----
-- Components Portmaps
-----

-- Gumnut
rst_i    <= '0';

gumnut          : gumnut_with_mem

PORT MAP(

    CLOCK_50,
    rst_i,
    port_cyc_o,
    port_stb_o,
    port_we_o,

```

```

                                '1',
                                port_adr_o,
                                port_dat_o,
                                port_dat_i,
                                int_req,
                                int_ack
                                );

-- Uart Port maps
uart_de10 : uart

                                PORT MAP(

                                CLOCK_50,
                                KEY_TEST,
                                tx_ena_de10,
                                tx_data_de10,
                                GPIO_24,
                                rx_busy_de10,
                                rx_error_de10,
                                rx_data_de10,
                                tx_busy_de10,
                                GPIO_25
                                );

-- Debounce key 1
button_de10 : debounce

                                PORT MAP(

                                CLOCK_50,
                                KEY(1),
                                key1_db
                                );

-- Debounce key 2
button2_de10 : debounce

                                PORT MAP(

                                CLOCK_50,
                                KEY(0),
                                key2_db
                                );

-- Accelerometer Portmaps
--Reset
reset : reset_delay PORT MAP( KEY_TEST, CLOCK_50, dly_rst );

--PLL
pll : spi_pll PORT MAP( dly_rst, CLOCK_50, spi_clk, spi_clk_out );

--Initial Setting and Data Read Back
spi_config : spi_ee_config PORT MAP( not dly_rst, spi_clk, spi_clk_out, GSENSOR_INT(0), data_x(7
DOWNT0 0),
                                data_x(15 DOWNT0 8), GSENSOR_SDI, GSENSOR_CS_N,
GSENSOR_SCLK );

--Led
led : led_driver PORT MAP( not dly_rst, CLOCK_50, data_x(9 DOWNT0 0), GSENSOR_INT(0), LEDR
);

-----
-- Display Process
-----

-- Process to update the display according to the data received
PROCESS(rx_data_de10)

    variable hundreds : integer range 0 to 9;

```

```

variable tens          : integer range 0 to 9;
variable single        : integer range 0 to 9;
variable int_value     : integer;

BEGIN

-- Save the received variable to integer
int_value := to_integer(unsigned(rx_data_de10(7 downto 0)));

-- Extracts hundreds, tens and single digit.
hundreds := int_value / 100;      -- Hundreds place
tens     := (int_value / 10) mod 10; -- Tens place
single   := int_value mod 10;      -- Single place

-- Assign display values based on digits

-- First segment
case single is
    when 0 => DISP1 <= "1000000"; -- 0
    when 1 => DISP1 <= "1111001"; -- 1
    when 2 => DISP1 <= "0100100"; -- 2
    when 3 => DISP1 <= "0110000"; -- 3
    when 4 => DISP1 <= "0011001"; -- 4
    when 5 => DISP1 <= "0010010"; -- 5
    when 6 => DISP1 <= "0000010"; -- 6
    when 7 => DISP1 <= "1111000"; -- 7
    when 8 => DISP1 <= "0000000"; -- 8
    when 9 => DISP1 <= "0010000"; -- 9
    when others => DISP1 <= "1000000"; -- 0 by default
end case;

-- Second segment
case tens is
    when 0 => DISP2 <= "1000000"; -- Blank leading zero
    when 1 => DISP2 <= "1111001"; -- 1
    when 2 => DISP2 <= "0100100"; -- 2
    when 3 => DISP2 <= "0110000"; -- 3
    when 4 => DISP2 <= "0011001"; -- 4
    when 5 => DISP2 <= "0010010"; -- 5
    when 6 => DISP2 <= "0000010"; -- 6
    when 7 => DISP2 <= "1111000"; -- 7
    when 8 => DISP2 <= "0000000"; -- 8
    when 9 => DISP2 <= "0010000"; -- 9
    when others => DISP2 <= "1000000"; -- 0 by default
end case;

-- Third segment
case hundreds is
    when 0 => DISP3 <= "1000000"; -- Blank leading zero
    when 1 => DISP3 <= "1111001"; -- 1
    when 2 => DISP3 <= "0100100"; -- 2
    when 3 => DISP3 <= "0110000"; -- 3
    when 4 => DISP3 <= "0011001"; -- 4
    when 5 => DISP3 <= "0010010"; -- 5
    when 6 => DISP3 <= "0000010"; -- 6
    when 7 => DISP3 <= "1111000"; -- 7
    when 8 => DISP3 <= "0000000"; -- 8
    when 9 => DISP3 <= "0010000"; -- 9
    when others => DISP3 <= "1000000"; -- 0 by default
end case;

END PROCESS;
-----

```

```

-- Gumnut code
-----

-- Output => TX_START -> data memory address: 0x00
PROCESS( CLOCK_50, rst_i )
BEGIN
    IF rst_i = '1' THEN
        tx_ena_de10 <= '1';
    ELSIF rising_edge( CLOCK_50 ) THEN
        IF port_adr_o = "00000000" and -- address port
           port_cyc_o = '1' and -- control
           port_stb_o = '1' and
           port_we_o = '1' --
        signals for I/O
        'write' operation
        THEN
            tx_ena_de10 <= port_dat_o(0);
        END IF;
    END IF;
END PROCESS;

-- Input <= KEY1 -> data memory address: 0x03
PROCESS( CLOCK_50, rst_i )
BEGIN
    IF rst_i = '1' THEN
        key1_domy <= ( OTHERS => '0' );
    ELSIF rising_edge( CLOCK_50 ) THEN
        -- Key1 (key1)
        IF port_adr_o = "00000011" and -- address port
           port_cyc_o = '1' and --
        control signals for I/O
           port_stb_o = '1' and
           port_we_o = '0'
        -- 'read' operation
        THEN
            key1_domy <= "0000000" & key1_db;
        END IF;
    END IF;
END PROCESS;

-- Input <= KEY0 -> data memory address: 0x04
PROCESS( CLOCK_50, rst_i )
BEGIN
    IF rst_i = '1' THEN
        key2_domy <= ( OTHERS => '0' );
    ELSIF rising_edge( CLOCK_50 ) THEN
        -- Key1 (key1)
        IF port_adr_o = "00000100" and -- address port
           port_cyc_o = '1' and --
        control signals for I/O
           port_stb_o = '1' and
           port_we_o = '0'
        -- 'read' operation
        THEN
            key2_domy <= "0000000" & key2_db;
        END IF;
    END IF;
END IF;

```

```

END PROCESS;

-- Input <= SW -> data memory address: 0x05
PROCESS( CLOCK_50, rst_i )
BEGIN
    IF rst_i = '1' THEN
        sw_domy <= ( OTHERS => '0' );
    ELSIF rising_edge( CLOCK_50 ) THEN

        -- Key1 (key1)
        IF port_adr_o = "00000101" and -- address port
            port_cyc_o = '1' and --
control signals for I/O
            port_stb_o = '1' and
            port_we_o = '0'
        THEN
            -- 'read' operation
            sw_domy <= SW(7 downto 0);
        END IF;
    END IF;
END PROCESS;

-- Input <= ACCE -> data memory address: 0x06
PROCESS( CLOCK_50, rst_i )
BEGIN
    IF rst_i = '1' THEN
        acce_domy <= ( OTHERS => '0' );
    ELSIF rising_edge( CLOCK_50 ) THEN

        -- Key1 (key1)
        IF port_adr_o = "00000110" and -- address port
            port_cyc_o = '1' and --
control signals for I/O
            port_stb_o = '1' and
            port_we_o = '0'
        THEN
            -- 'read' operation
            acce_domy <= LEDR(8 downto 1);
        END IF;
    END IF;
END PROCESS;

-- Output => TX_DATA -> data memory address: 0x02
PROCESS( CLOCK_50, rst_i )
BEGIN
    IF rst_i = '1' THEN
        tx_data_de10 <= ( OTHERS => '0' );
    ELSIF rising_edge( CLOCK_50 ) THEN
        IF port_adr_o = "00000010" and -- address port
            port_cyc_o = '1' and -- control
signals for I/O
            port_stb_o = '1' and
            port_we_o = '1'
        THEN
            -- 'write' operation
            tx_data_de10 <= port_dat_o;
        END IF;
    END IF;
END PROCESS;

```

```

END PROCESS;

-- Select procedure to send the correct input data to Gumnut
WITH port_adr_o SELECT

    port_dat_i <= key1_domy WHEN "00000011",
                                key2_domy WHEN "00000100",
                                sw_domy   WHEN "00000101",
                                acce_domy WHEN "00000110",
                                UNAFFECTED WHEN OTHERS;

END Structural;

```

B. Código Assembly

Python

```

                                text
                                org 0x000 ; start here on reset
                                jmp main

; Data memory layout

                                data
TX_START: bss 1                ;0
tx_disable: byte 1             ;1
TX_DATA:  bss 1                ;2
KEY1:      bss 1                ;3
KEY2:      bss 1                ;4
SW:        bss 1                ;5

```

```

ACCE:                bss 1                ;6
KEY1_ON:  byte 1                ;key1 on  = 1
KEY1_OFF: byte 2                ;key1 off  = 2
KEY2_ON:  byte 3                ;key2 on  = 3
KEY2_OFF: byte 4                ;key2 off  = 4
SW1_ON:   byte 9                ;sw1 on   = 9
SW1_OFF:  byte 10               ;sw1 off  = 10
SW2_ON:   byte 11               ;sw2 on   = 11
SW2_OFF:  byte 12               ;sw2 off  = 12
ACCE_OFF1: byte 6                ;acce off = 6
ACCE_OFF2: byte 8                ;acce off = 8
ACCE_R:   byte 7                ;acce right = 7
ACCE_L:   byte 5                ;acce left  = 5

; Main program

text
org 0x010

main:                ldm r1, r0                ;r1 = 0
                    inp r1, KEY1                ;r1 = key1
                    jsb delay                ;delay
                    and r1, r1, 1                ;r1 = r1 and 1
                    bz not_key1                ;if r1 = 0 jump to not_key1
                    ldm r1, r0                ;if !r1 = 0 then r1 = 0
                    ldm r2, KEY1_ON            ;r2 = key1 on
                    out r2, TX_DATA            ;Data = r2
                    out r0, TX_START            ;Start = 0
                    ldm r2, tx_disable         ;r2 = 1
                    out r2, TX_START            ;Start = 1
                    jmp next_sw1                ;jumps

;when key1 is not pressed
not_key1:            ldm r1, r0                ;r1 = 0
                    ldm r2, KEY1_OFF            ;r2 = key1 off
                    out r2, TX_DATA            ;Data = r2
                    out r0, TX_START            ;Start = 0
                    ldm r2, tx_disable         ;r2 = 1
                    out r2, TX_START            ;start = 1
                    jmp next_sw1                ;jumps

;check for swith 1
next_sw1:            ldm r1, r0                ;r1 = 0
                    inp r1, SW                ;r1 = swithes
                    jsb delay                ;dealy
                    and r1, r1, 1                ;r1 = r1 and 1
                    bz not_sw1                ;if r1 = 0
                    ldm r1, r0                ;if r1 = 1
                    ldm r2, SW1_ON            ;r2 = sw1 on
                    out r2, TX_DATA            ;Data = r2
                    out r0, TX_START            ;start = 0
                    ldm r2, tx_disable         ;r2 = 1
                    out r2, TX_START            ;start = 1
                    jmp next_sw2                ;jumps

;when sw1 is off
not_sw1:             ldm r1, r0                ;r1 = 0
                    ldm r2, SW1_OFF            ;r2 = sw1 off
                    out r2, TX_DATA            ;Data = r2
                    out r0, TX_START            ;start = 0
                    ldm r2, tx_disable         ;r2 = 1
                    out r2, TX_START            ;strat = 1
                    jmp next_sw2                ;jumps

;check for switch 2
next_sw2:            ldm r1, r0                ;r1 = 0
                    inp r1, SW                ;r1 = swithes

```

```

        jsb delay                ;delay
        and r1, r1, 2            ;r1 = r1 and 2
        bz not_sw2              ;if r1 = 0
        ldm r1, r0               ;if r1 = 1
        ldm r2, SW2_ON           ;r2 = sw2 on
        out r2, TX_DATA          ;data = r2
        out r0, TX_START         ;start = 0
        ldm r2, tx_disable       ;r2 = 1
        out r2, TX_START         ;start = 1
        jmp next_key2            ;jumps

;when sw2 is not on
not_sw2: ldm r1, r0              ;r1 = 0
        ldm r2, SW2_OFF         ;r2 = sw2 off
        out r2, TX_DATA         ;data = r2
        out r0, TX_START        ;start = 0
        ldm r2, tx_disable       ;r2 = 1
        out r2, TX_START        ;start = 1
        jmp next_key2           ;jumps

;check for key2
next_key2: ldm r1, r0            ;r1 = 0
        inp r1, KEY2            ;r1 = key2
        jsb delay              ;delay
        and r1, r1, 1          ;r1 = r1 and 1
        bz not_key2            ;r1 = 0
        ldm r1, r0             ;if r1 = 1
        ldm r2, KEY2_ON        ;r2 = key2 on
        out r2, TX_DATA        ;data = r2
        out r0, TX_START       ;start = 0
        ldm r2, tx_disable      ;r2 = 1
        out r2, TX_START       ;start = 1
        jmp next_acce          ;jumps

;when key 2 is not pressed
not_key2: ldm r1, r0            ;r1 = 0
        ldm r2, KEY2_OFF       ;r2 = key2 off
        out r2, TX_DATA        ;data = r2
        out r0, TX_START       ;start = 0
        ldm r2, tx_disable      ;r2 = 1
        out r2, TX_START       ;start = 1
        jmp next_acce          ;jumps

;check for accelerometer
next_acce: ldm r1, r0           ;r1 = 0
        inp r1, ACCE           ;r1 = acce
        jsb delay              ;delay
        and r3, r1, 24         ;r3 = r1 and 00011000
        bnz not_acce           ;if acce = middle

        and r3, r1, 1          ;r3 = r1 and 00000001
        bnz acce_right         ;if acce = right
        and r3, r1, 2          ;r3 = r1 and 00000010
        bnz acce_right         ;if acce = right
        and r3, r1, 4          ;r3 = r1 and 00000100
        bnz acce_right         ;if acce = right

        and r3, r1, 32         ;r3 = r1 and 00100000
        bnz acce_left          ;if acce = left
        and r3, r1, 64         ;r3 = r1 and 01000000
        bnz acce_left          ;if acce = left
        and r3, r1, 128        ;r3 = r1 and 10000000
        bnz acce_left          ;if acce = left

        ldm r1, r0             ;r1 = 0
        ldm r2, ACCE_OFF       ;r2 = acce off

```

```

        out r2, TX_DATA          ;data = r2
        out r0, TX_START        ;start = 0
        ldm r2, tx_disable      ;r2 = 1
        out r2, TX_START        ;start = 1
        jsb delay                ;delay
        jmp main                ;loops back to main

;when acce is in the middle
not_acce:    ldm r1, r0          ;r1 = 0
            ldm r2, ACCE_OFF1    ;r2 = acce off 1
            out r2, TX_DATA      ;data = r2
            out r0, TX_START     ;start = 0
            ldm r2, tx_disable   ;r2 = 1
            out r2, TX_START     ;start = 1
            jsb delay            ;delay
            ldm r1, r0          ;r1 = 0
            ldm r2, ACCE_OFF2    ;r2 = acce off 2
            out r2, TX_DATA      ;data = r2
            out r0, TX_START     ;start = 0
            ldm r2, tx_disable   ;r2 = 1
            out r2, TX_START     ;start = 1
            jsb delay            ;delay
            jmp main            ;loops back to main

;when acce is tilt right
acce_right:  ldm r1, r0          ;r1 = r0
            ldm r2, ACCE_R        ;r2 = acce right
            out r2, TX_DATA      ;data = r2
            out r0, TX_START     ;start = 0
            ldm r2, tx_disable   ;r2 = 1
            out r2, TX_START     ;start = 1
            jsb delay            ;delay
            jmp main            ;loops back to main

;when acce is tilt left
acce_left:   ldm r1, r0          ;r1 = r0
            ldm r2, ACCE_L        ;r2 = acce left
            out r2, TX_DATA      ;data = r2
            out r0, TX_START     ;start = 0
            ldm r2, tx_disable   ;r2 = 1
            out r2, TX_START     ;start = 1
            jsb delay            ; delay
            jmp main            ;loops back to main

;delay
delay:       add r7, r0, 0
again3:      add r6, r0, 0
again2:      add r5, r0, 0
again1:      add r5, r5, 1
            sub r0, r5, 0xFF
            bnz again1
            add r6, r6, 1
            sub r0, r6, 0xFF
            bnz again2
            add r7, r7, 1
            sub r0, r7, 0x0C
            bnz again3
            ret

```

C. Codigo Unity

Este código hace la conexión serial, el movimiento del tractor mediante la conexión serial y el constante envío de datos en bits al igual que el muestreo de datos de recolección del maíz para su demostración en el display

C/C++

```
using UnityEngine;  
using System.IO.Ports;
```

```

public class PrometeoCarController : MonoBehaviour
{
    public int maxSpeed = 10;
    public int maxReverseSpeed = 15;
    public float accelerationMultiplier = 0.5f;
    public int maxSteeringAngle = 45;
    public float steeringSpeed = 10f;

    public GameObject frontLeftMesh;
    public WheelCollider frontLeftCollider;
    public GameObject frontRightMesh;
    public WheelCollider frontRightCollider;
    public GameObject rearLeftMesh;
    public WheelCollider rearLeftCollider;
    public GameObject rearRightMesh;
    public WheelCollider rearRightCollider;

    private Rigidbody carRigidbody;
    private float steeringAxis;
    private float throttleAxis;
    private SerialPort serialPort;
    private bool switch0Activated = false;
    private bool switch1Activated = false;
    private bool switch2Activated = false;
    private bool switch3Activated = false;
    // Botones para acelerar y para frenar
    private bool button1Activated = false;
    private bool button1Deactivated = false;
    private bool button2Activated = false;
    private bool button2Deactivate = false;
    // con esta instancio corn collector para poderla mandar a llamar en este
script
    private CornCollector cornCollector; // Referencia al script CornCollector

    void Start()
    {
        carRigidbody = gameObject.GetComponent<Rigidbody>();
        carRigidbody.centerOfMass = new Vector3(0, -0.9f, 0);

        serialPort = new SerialPort("COM5", 115200);
        // con esta funcoin al empezar mando a llamar a corn collector
        cornCollector = FindObjectOfType<CornCollector>();
        try
        {
            serialPort.Open();

```

```

        serialPort.ReadTimeout = 1;
    }
    catch (System.Exception e)
    {
        Debug.LogError("Error al abrir el puerto serial: " + e.Message);
    }
}

void Update()
{
    ControlVehicleWithSerial();
    AnimateWheelMeshes();
    // Obtener la cantidad de maíz recolectado
    int cornCollected = cornCollector.GetCorn();
    // Convertir la cantidad de maíz recolectado a byte
    byte[] cornData = { (byte)cornCollected };
    // Enviar la cantidad de maíz recolectado a través del puerto serie
    SendDataToSerialPort(cornData);
}

void SendDataToSerialPort(byte[] data)
{
    if (serialPort != null && serialPort.IsOpen)
    {
        try
        {
            serialPort.Write(data, 0, data.Length);
        }
        catch (System.Exception e)
        {
            Debug.LogWarning("Error al escribir datos en el puerto serial: " +
e.Message);
        }
    }
}

void ControlVehicleWithSerial()
{
    if (serialPort != null && serialPort.IsOpen)
    {
        try
        {
            if (serialPort.BytesToRead > 0)
            {

```

```

        byte[] data = new byte[1];
        serialPort.Read(data, 0, 1);
        ParseSerialData(data[0]);
    }
}
catch (System.Exception e)
{
    Debug.LogWarning("Error al leer datos del puerto serial: " +
e.Message);
}
}

void ParseSerialData(byte data)
{
    Debug.Log("Dato recibido desde el puerto serial: " + data);

    if (data == 0x01) // Switch 0 arriba
    {
        switch0Activated = true;
        if (!switch1Activated)
        {
            GoForward();
        }
    }
    else if (data == 0x02) // Switch 0 abajo
    {
        switch0Activated = false;
        throttleAxis = 0;
        ApplyMotorTorque(throttleAxis);
    }
    else if (data == 0x03) // Switch 1 arriba
    {
        switch1Activated = true;
        if (!switch0Activated)
        {
            GoReverse();
        }
    }
    else if (data == 0x04) // Switch 1 abajo
    {
        switch1Activated = false;
        throttleAxis = 0;
        ApplyMotorTorque(throttleAxis);
    }
}

```

```

else if (data == 0x05) // Switch 2 arriba
{
    switch2Activated = true;
    TurnLeft();
}
else if (data == 0x06) // Switch 2 abajo
{
    switch2Activated = false;
    steeringAxis = 0;
    ApplySteering();
}
else if (data == 0x07) // Switch 3 arriba
{
    switch3Activated = true;
    TurnRight();
}
else if (data == 0x08) // Switch 3 abajo
{
    switch3Activated = false;
    steeringAxis = 0;
    ApplySteering();
}

// Esto es lo nuevo
else if (data == 0x09) // boton 1
{
    button1Activated = true;
    boost();
}
else if (data == 0x0A)
{
    button1Deactivated = false;
    DeactivateBoost();
}

else if (data == 0x0B) // boton 2
{
    button2Activated = true;
    breaktractor();
}
else if (data == 0x0C)
{
    button2Deactivate = false;
    DeactivateBreakTractor();
}

```

```

    }
}

public void boost(){
    if(button1Activated){
        // Aca tiene que ir la condiciones para que se realice
        float boostMultiplier = 4.0f; // Esto seria el doble de velocidad
        ApplyMotorTorque(boostMultiplier); //Aca aplicamos el incremento del
boostmultiplier
        Debug.Log("Activacion de boost");
    }
}

public void DeactivateBoost(){
    ThrottleOff();
}

public void breaktractor(){
    if(button2Activated){
        // Aca voy a activar las condiciones para que se realice
        // float brakeMultiplier = 0.0f; // Aca se pararia por completo
        // ApplyMotorTorque(-brakeMultiplier); //Aplicar de desendencia del
torque
        ApplyMotorTorque(0.0f);
        Debug.Log("Freno");
    }
}

public void DeactivateBreakTractor(){
    ThrottleOff();
}

public void TurnLeft()
{
    if (switch2Activated && !switch3Activated)
    {
        steeringAxis = -1f;
        ApplySteering();
    }
}

public void TurnRight()
{
    if (switch3Activated && !switch2Activated)

```

```

    {
        steeringAxis = 1f;
        ApplySteering();
    }
}

void ApplySteering()
{
    var steeringAngle = steeringAxis * maxSteeringAngle;
    frontLeftCollider.steerAngle = steeringAngle;
    frontRightCollider.steerAngle = steeringAngle;
}

public void GoReverse()
{
    if (switch1Activated && !switch0Activated)
    {
        throttleAxis = -1f;
        ApplyMotorTorque(throttleAxis);
    }
}

public void GoForward()
{
    if (switch0Activated && !switch1Activated)
    {
        throttleAxis = 3f;
        ApplyMotorTorque(throttleAxis);
    }
}

void ApplyMotorTorque(float throttle)
{
    float torque = accelerationMultiplier * 50f * throttle;
    frontLeftCollider.motorTorque = torque;
    frontRightCollider.motorTorque = torque;
    rearLeftCollider.motorTorque = torque;
    rearRightCollider.motorTorque = torque;
}

void ThrottleOff()
{
    throttleAxis = 0;
    frontLeftCollider.motorTorque = 0;
    frontRightCollider.motorTorque = 0;
}

```

```

        rearLeftCollider.motorTorque = 0;
        rearRightCollider.motorTorque = 0;
    }

    void AnimateWheelMeshes()
    {
        UpdateWheelPose(frontLeftCollider, frontLeftMesh);
        UpdateWheelPose(frontRightCollider, frontRightMesh);
        UpdateWheelPose(rearLeftCollider, rearLeftMesh);
        UpdateWheelPose(rearRightCollider, rearRightMesh);
    }

    void UpdateWheelPose(WheelCollider collider, GameObject mesh)
    {
        Vector3 pos;
        Quaternion rot;
        collider.GetWorldPose(out pos, out rot);
        mesh.transform.position = pos;
        mesh.transform.rotation = rot;
    }

    void OnDestroy()
    {
        if (serialPort != null && serialPort.IsOpen)
        {
            serialPort.Close();
        }
    }
}

```